

Line-by-Line Source Code Explanation

Full technical walkthrough of the Streamlit frontend entry point, every executable and non-executable line documented in order.

TARGET FILE app.py	TOTAL LINES 976	LANGUAGE Python 3.x (Streamlit)	DOMAIN Frontend / Web Application UI
-----------------------	--------------------	------------------------------------	---

#	SOURCE CODE	TECHNICAL EXPLANATION
1	<code>import sys</code>	Imports Python's built-in <code>sys</code> module, used to access interpreter/path internals.
2	<code>import os</code>	Imports Python's built-in <code>os</code> module, used to work with the filesystem and OS paths.
3		Blank line — vertical spacing added purely for readability; has no functional effect.
4	<code>sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))</code>	Adds the project's parent directory to Python's module search path, so the sibling <code>agent</code> package (analyzer/outreach/billing) can be imported below.
5		Blank line — vertical spacing added purely for readability; has no functional effect.
6	<code>import streamlit as st</code>	Imports the <code>**Streamlit**</code> library and aliases it as <code>st</code> — this is the core frontend framework used throughout the file to render every widget, layout and page element in the browser.
7	<code>import json</code>	Imports Python's built-in <code>json</code> module, used to read/write JSON data files.
8	<code>import time</code>	Imports Python's built-in <code>time</code> module, used to generate timestamps and control delays.
9	<code>import re</code>	Imports Python's built-in <code>re</code> module, used to validate and parse text with regular expressions.
10	<code>import io</code>	Imports Python's built-in <code>io</code> module, used to handle in-memory byte streams (e.g. QR code images).
11	<code>import secrets</code>	Imports Python's built-in <code>secrets</code> module, used to generate cryptographically secure random tokens (trial user IDs).
12	<code>import shutil</code>	Imports Python's built-in <code>shutil</code> module, used to perform high-level file operations.
13	<code>import datetime</code>	Imports Python's built-in <code>datetime</code> module, used to work with calendar dates and times.
14	<code>import pandas as pd</code>	Imports the <code>**pandas**</code> data-analysis library as <code>pd</code> , used later to build the scraped-leads table shown with <code>st.dataframe</code> .
15	<code>from fpdf import FPDF</code>	Imports <code>FPDF</code> from the local <code>fpdf</code> module for use in this file.
16	<code>from agent.analyzer import DetectiveAgent</code>	Imports <code>DetectiveAgent</code> from the local <code>agent.analyzer</code> module for use in this file.
17	<code>from agent.outreach import (</code>	Begins a multi-line import that pulls several names out of the local <code>agent.outreach</code> module (the list continues on the following indented lines).
18	<code> scrape_leads_sync,</code>	Names <code>scrape_leads_sync</code> as one of the items being imported by the multi-line import statement opened above.
19	<code> send_cold_emails,</code>	Names <code>send_cold_emails</code> as one of the items being imported by the multi-line import statement opened above.
20	<code> load_leads,</code>	Names <code>load_leads</code> as one of the items being imported by the multi-line import statement opened above.
21	<code> save_leads,</code>	Names <code>save_leads</code> as one of the items being imported by the multi-line import statement opened above.
22	<code> COLD_EMAIL_SUBJECT,</code>	Names <code>COLD_EMAIL_SUBJECT</code> as one of the items being imported by the multi-line import statement opened above.
23	<code> COLD_EMAIL_BODY,</code>	Names <code>COLD_EMAIL_BODY</code> as one of the items being imported by the multi-line import statement opened above.
24	<code> LEADS_FILE,</code>	Names <code>LEADS_FILE</code> as one of the items being imported by the multi-line import statement opened above.
25	<code>)</code>	Closes the multi-line <code>from agent.outreach import (...)</code> opened above.
26	<code>from agent.billing import (</code>	Begins a multi-line import that pulls several names out of the local <code>agent.billing</code> module (the list continues on the following indented lines).
27	<code> submit_payment,</code>	Names <code>submit_payment</code> as one of the items being imported by the multi-line import statement opened above.
28	<code> submit_topup,</code>	Names <code>submit_topup</code> as one of the items being imported by the multi-line import statement opened above.
29	<code> approve_payment,</code>	Names <code>approve_payment</code> as one of the items being imported by the multi-line import statement opened above.
30	<code> flag_partial,</code>	Names <code>flag_partial</code> as one of the items being imported by the multi-line import statement opened above.
31	<code> get_pending_payments,</code>	Names <code>get_pending_payments</code> as one of the items being imported by the multi-line import statement opened above.
32	<code> get_partial_by_utr,</code>	Names <code>get_partial_by_utr</code> as one of the items being imported by the multi-line import statement opened above.
33	<code> STATUS_PENDING,</code>	Names <code>STATUS_PENDING</code> as one of the items being imported by the multi-line import statement opened above.
34	<code> STATUS_APPROVED,</code>	Names <code>STATUS_APPROVED</code> as one of the items being imported by the multi-line import statement opened above.
35	<code> STATUS_PARTIAL,</code>	Names <code>STATUS_PARTIAL</code> as one of the items being imported by the multi-line import statement opened above.
36	<code> STATUS_TOPUP_DONE,</code>	Names <code>STATUS_TOPUP_DONE</code> as one of the items being imported by the multi-line import statement opened above.
37	<code> STATUS_FLAGGED,</code>	Names <code>STATUS_FLAGGED</code> as one of the items being imported by the multi-line import statement opened above.
38	<code>)</code>	Closes the multi-line <code>from agent.billing import (...)</code> opened above.
39		Blank line — vertical spacing added purely for readability; has no functional effect.
40	<code>st.set_page_config(</code>	Configures the browser tab for the whole app — sets the page title, favicon/emoji icon and switches to Streamlit's wide layout so more content fits horizontally on screen.
41	<code> page_title="Detective Agentic AI - Criminal Profiler",</code>	Continuation of the multi-line <code>st.set_page_config(...)</code> call above, contributing the text “Detective Agentic AI - Criminal Profiler”.
42	<code> page_icon="🕵️",</code>	Continuation of the multi-line <code>st.set_page_config(...)</code> call above, contributing the text “🕵️”.
43	<code> layout="wide"</code>	Continuation of the multi-line <code>st.set_page_config(...)</code> call above, contributing the text “wide”.
44	<code>)</code>	Closes the multi-line <code>st.set_page_config(...)</code> call opened above.
45		Blank line — vertical spacing added purely for readability; has no functional effect.
46	<code>PLATFORM_URL = "https://skilluphackathon2026-crvfgw9pkgzk3bzrmhwmfq.streamlit.app/?trial_user=GokxfJ7NYPEF0Ej2e2ZJXhF9DB409cZb"</code>	Defines <code>PLATFORM_URL</code> — the public URL of the deployed app, used inside outreach emails.
47		Blank line — vertical spacing added purely for readability; has no functional effect.
48	<code>DATA_DIR = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "data"))</code>	Defines <code>DATA_DIR</code> — the absolute filesystem path to the app's <code>data</code> folder.
49	<code>def generate_email_content(lead_id, email_type):</code>	Defines <code>generate_email_content</code> — a function that takes a <code>lead_id</code> and <code>email_type</code> to return a string of HTML content for an email.

49	<code>PAYMENTS_FILE = os.path.join(DATA_DIR, "payments.json")</code>	Defines <code>PAYMENTS_FILE</code> — the path to the JSON file that stores submitted payment records.
50	<code>TRIAL_USAGE_FILE = os.path.join(DATA_DIR, "trial_usage.json")</code>	Defines <code>TRIAL_USAGE_FILE</code> — the path to the JSON file that tracks free-trial usage per visitor.
51	<code>TRIAL_LIMIT = 25</code>	Defines <code>TRIAL_LIMIT</code> — the maximum number of free evaluations a trial user gets.
52		Blank line — vertical spacing added purely for readability; has no functional effect.
53	<code>def _ensure_data():</code>	Defines the function <code>_ensure_data(...)</code> , which makes sure the local <code>data/</code> folder exists before any file is read or written.
54	<code>os.makedirs(DATA_DIR, exist_ok=True)</code>	Calls <code>os.makedirs(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
55		Blank line — vertical spacing added purely for readability; has no functional effect.
56	<code>def load_payments():</code>	Defines the function <code>load_payments(...)</code> , which reads all previously submitted payment records from disk.
57	<code>_ensure_data()</code>	Calls <code>_ensure_data(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
58	<code>if not os.path.exists(PAYMENTS_FILE):</code>	Conditional check — runs the indented block below only when <code>not os.path.exists(PAYMENTS_FILE)</code> evaluates to true.
59	<code>return []</code>	Returns <code>[]</code> as the result of the current function, ending its execution.
60	<code>try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
61	<code>with open(PAYMENTS_FILE, "r", encoding="utf-8") as f:</code>	Opens a file (<code>'open(PAYMENTS_FILE, "r", encoding="utf-8") as f'</code>) for reading/writing; the file is automatically closed when the indented block ends.
62	<code>return json.load(f)</code>	Returns <code>json.load(f)</code> as the result of the current function, ending its execution.
63	<code>except Exception:</code>	Catches an error from the <code>try</code> block above of type <code>Exception</code> , so the app can show a friendly message instead of crashing.
64	<code>return []</code>	Returns <code>[]</code> as the result of the current function, ending its execution.
65		Blank line — vertical spacing added purely for readability; has no functional effect.
66	<code>def save_payments(payments):</code>	Defines the function <code>save_payments(...)</code> , which writes the payments list back to disk as JSON.
67	<code>_ensure_data()</code>	Calls <code>_ensure_data(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
68	<code>with open(PAYMENTS_FILE, "w", encoding="utf-8") as f:</code>	Opens a file (<code>'open(PAYMENTS_FILE, "w", encoding="utf-8") as f'</code>) for reading/writing; the file is automatically closed when the indented block ends.
69	<code>json.dump(payments, f, indent=2, ensure_ascii=False)</code>	Calls <code>json.dump(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
70		Blank line — vertical spacing added purely for readability; has no functional effect.
71	<code>def _get_trial_user_id() -> str:</code>	Defines the function <code>_get_trial_user_id(...)</code> , which gets (or creates) a stable anonymous ID for the current visitor's browser, used to track their free-trial usage.
72	<code>"""Keep a stable browser identifier across Streamlit reruns and refreshes."""</code>	Executes: <code>"""Keep a stable browser identifier across Streamlit reruns and refreshes."""</code> — supporting statement within the current block.
73	<code>try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
74	<code>current_id = str(st.query_params.get("trial_user", "")).strip()</code>	Inside the current function/block, sets the variable <code>current_id</code> to the result of <code>'str(st.query_params.get("trial_user", "")).strip()'</code> .
75	<code>except Exception:</code>	Catches an error from the <code>try</code> block above of type <code>Exception</code> , so the app can show a friendly message instead of crashing.
76	<code>current_id = ""</code>	Inside the current function/block, sets the variable <code>current_id</code> to the result of <code>''</code> .
77		Blank line — vertical spacing added purely for readability; has no functional effect.
78	<code>if current_id and re.fullmatch(r"[A-Za-z0-9_]{20,100}", current_id):</code>	Conditional check — runs the indented block below only when <code>'current_id and re.fullmatch(r"[A-Za-z0-9_]{20,100}", current_id)'</code> evaluates to true.
79	<code>return current_id</code>	Returns <code>current_id</code> as the result of the current function, ending its execution.
80		Blank line — vertical spacing added purely for readability; has no functional effect.
81	<code>new_id = secrets.token_urlsafe(24)</code>	Inside the current function/block, sets the variable <code>new_id</code> to the result of <code>secrets.token_urlsafe(24)</code> .
82	<code>st.query_params["trial_user"] = new_id</code>	Inside the current function/block, sets the variable <code>'st.query_params["trial_user"]</code> to the result of <code>new_id</code> .
83	<code>st.rerun()</code>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
84	<code>return new_id</code>	Returns <code>new_id</code> as the result of the current function, ending its execution.
85		Blank line — vertical spacing added purely for readability; has no functional effect.
86	<code>def _load_trial_usage():</code>	Defines the function <code>_load_trial_usage(...)</code> , which loads the trial-usage counters for all visitors from disk.
87	<code>_ensure_data()</code>	Calls <code>_ensure_data(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
88	<code>if not os.path.exists(TRIAL_USAGE_FILE):</code>	Conditional check — runs the indented block below only when <code>not os.path.exists(TRIAL_USAGE_FILE)</code> evaluates to true.
89	<code>return {}</code>	Returns <code>{}</code> as the result of the current function, ending its execution.
90	<code>try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
91	<code>with open(TRIAL_USAGE_FILE, "r", encoding="utf-8") as f:</code>	Opens a file (<code>'open(TRIAL_USAGE_FILE, "r", encoding="utf-8") as f'</code>) for reading/writing; the file is automatically closed when the indented block ends.
92	<code>data = json.load(f)</code>	Inside the current function/block, sets the variable <code>data</code> to the result of <code>json.load(f)</code> .
93	<code>return data if isinstance(data, dict) else {}</code>	Returns <code>data if isinstance(data, dict) else {}</code> as the result of the current function, ending its execution.
94	<code>except (OSError, json.JSONDecodeError, TypeError):</code>	Catches an error from the <code>try</code> block above of type <code>(OSError, json.JSONDecodeError, TypeError)</code> ..., so the app can show a friendly message instead of crashing.
95	<code>return {}</code>	Returns <code>{}</code> as the result of the current function, ending its execution.
96		Blank line — vertical spacing added purely for readability; has no functional effect.
97	<code>def _save_trial_usage(usage):</code>	Defines the function <code>_save_trial_usage(...)</code> , which persists updated trial-usage counters to disk.
98	<code>_ensure_data()</code>	Calls <code>_ensure_data(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
99	<code>with open(TRIAL_USAGE_FILE, "w", encoding="utf-8") as f:</code>	Opens a file (<code>'open(TRIAL_USAGE_FILE, "w", encoding="utf-8") as f'</code>) for reading/writing; the file is automatically closed when the indented block ends.
100	<code>json.dump(usage, f, indent=2, ensure_ascii=False)</code>	Calls <code>json.dump(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
101		Blank line — vertical spacing added purely for readability; has no functional effect.
102	<code>def _trial_remaining(user_id: str) -> int:</code>	Defines the function <code>_trial_remaining(...)</code> , which calculates how many free evaluations a given trial user has left.
103	<code>record = load_trial_usage().get(user_id, {})</code>	Inside the current function/block, sets the variable <code>record</code> to the result of <code>load_trial_usage().get(user_id, {})</code> .

104	<code>try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
105	<code>used = max(0, int(record.get("used", 0)))</code>	Inside the current function/block, sets the variable <code>used</code> to the result of <code>`max(0, int(record.get("used", 0)))`</code> .
106	<code>except (AttributeError, TypeError, ValueError):</code>	Catches an error from the <code>try</code> block above of type <code>(AttributeError, TypeError, ValueError)</code> , so the app can show a friendly message instead of crashing.
107	<code>used = 0</code>	Inside the current function/block, sets the variable <code>used</code> to the result of <code>0</code> .
108	<code>return max(0, TRIAL_LIMIT - min(TRIAL_LIMIT, used))</code>	Returns <code>max(0, TRIAL_LIMIT - min(TRIAL_LIMIT, used))</code> as the result of the current function, ending its execution.
109		Blank line — vertical spacing added purely for readability; has no functional effect.
110	<code>def _consume_trial(user_id: str) -> int:</code>	Defines the function <code>_consume_trial(...)</code> , which deducts one evaluation from a trial user's remaining quota and saves it.
111	<code>usage = _load_trial_usage()</code>	Inside the current function/block, sets the variable <code>usage</code> to the result of <code>_load_trial_usage()</code> .
112	<code>record = usage.get(user_id, {})</code>	Inside the current function/block, sets the variable <code>record</code> to the result of <code>usage.get(user_id, {})</code> .
113	<code>try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
114	<code>used = max(0, int(record.get("used", 0)))</code>	Inside the current function/block, sets the variable <code>used</code> to the result of <code>`max(0, int(record.get("used", 0)))`</code> .
115	<code>except (AttributeError, TypeError, ValueError):</code>	Catches an error from the <code>try</code> block above of type <code>(AttributeError, TypeError, ValueError)</code> , so the app can show a friendly message instead of crashing.
116	<code>used = 0</code>	Inside the current function/block, sets the variable <code>used</code> to the result of <code>0</code> .
117		Blank line — vertical spacing added purely for readability; has no functional effect.
118	<code>if used >= TRIAL_LIMIT:</code>	Conditional check — runs the indented block below only when <code>`used >= TRIAL_LIMIT`</code> evaluates to true.
119	<code>return 0</code>	Returns <code>0</code> as the result of the current function, ending its execution.
120		Blank line — vertical spacing added purely for readability; has no functional effect.
121	<code>usage[user_id] = {</code>	Inside the current function/block, sets the variable <code>usage[user_id]</code> to the result of <code>{</code> .
122	<code>"used": used + 1,</code>	Supplies the <code>used</code> entry (<code>used + 1,`</code> ) as part of the multi-line <code>usage[user_id]</code> assignment` started above.
123	<code>"updated_at": time.strftime("%Y-%m-%d %H:%M:%S"),</code>	Supplies the <code>updated_at</code> entry (<code>`time.strftime("%Y-%m-%d %H:%M:%S"),`</code>) as part of the multi-line <code>usage[user_id]</code> assignment` started above.
124	<code>}</code>	Closes the multi-line <code>usage[user_id]</code> assignment` opened above.
125	<code>_save_trial_usage(usage)</code>	Calls <code>_save_trial_usage(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
126	<code>return TRIAL_LIMIT - used - 1</code>	Returns <code>TRIAL_LIMIT - used - 1</code> as the result of the current function, ending its execution.
127		Blank line — vertical spacing added purely for readability; has no functional effect.
128	<code>ADMIN_PIN = "739"</code>	Defines <code>ADMIN_PIN</code> — the fallback 3-digit PIN used to unlock the admin portal.
129		Blank line — vertical spacing added purely for readability; has no functional effect.
130	<code>def _get_admin_pin() -> str:</code>	Defines the function <code>_get_admin_pin(...)</code> , which resolves the admin login PIN, preferring a value from Streamlit secrets over the hard-coded default.
131	<code>try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
132	<code>configured_pin = str(st.secrets["ADMIN_PIN"]).strip()</code>	Inside the current function/block, sets the variable <code>configured_pin</code> to the result of <code>`str(st.secrets["ADMIN_PIN"]).strip()`</code> .
133	<code>if re.fullmatch(r"d{3}", configured_pin):</code>	Conditional check — runs the indented block below only when <code>`re.fullmatch(r"d{3}", configured_pin)`</code> evaluates to true.
134	<code>return configured_pin</code>	Returns <code>configured_pin</code> as the result of the current function, ending its execution.
135	<code>except Exception:</code>	Catches an error from the <code>try</code> block above of type <code>Exception</code> , so the app can show a friendly message instead of crashing.
136	<code>pass</code>	No-op placeholder — deliberately does nothing (satisfies Python's block-body requirement).
137	<code>return ADMIN_PIN</code>	Returns <code>ADMIN_PIN</code> as the result of the current function, ending its execution.
138		Blank line — vertical spacing added purely for readability; has no functional effect.
139	<code>def _get_gmail_app_password() -> str:</code>	Defines the function <code>_get_gmail_app_password(...)</code> , which reads the Gmail App Password used for sending outreach emails from Streamlit secrets.
140	<code>"""Read the Gmail App Password from Streamlit secrets."""</code>	Executes: <code>"""Read the Gmail App Password from Streamlit secrets."""</code> — supporting statement within the current block.
141	<code>for key in ("GMAIL_APP_PASSWORD", "EMAIL_APP_PASSWORD", "GMAIL_PASSWORD"):</code>	Loop — iterates <code>key</code> over <code>`("GMAIL_APP_PASSWORD", "EMAIL_APP_PASSWORD", "GMAIL_PASSWOR..."`</code> , running the indented block once per item.
142	<code>try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
143	<code>value = st.secrets[key]</code>	Inside the current function/block, sets the variable <code>value</code> to the result of <code>st.secrets[key]</code> .
144	<code>if value:</code>	Conditional check — runs the indented block below only when <code>value</code> evaluates to true.
145	<code>return str(value).strip()</code>	Returns <code>str(value).strip()</code> as the result of the current function, ending its execution.
146	<code>except Exception:</code>	Catches an error from the <code>try</code> block above of type <code>Exception</code> , so the app can show a friendly message instead of crashing.
147	<code>pass</code>	No-op placeholder — deliberately does nothing (satisfies Python's block-body requirement).
148	<code>return ""</code>	Returns <code>""</code> as the result of the current function, ending its execution.
149		Blank line — vertical spacing added purely for readability; has no functional effect.
150	<code>_ss_defaults = {</code>	Starts the <code>_ss_defaults</code> dictionary, listing every piece of persistent UI state (quota counters, admin flag, current results, dialog visibility, etc.) and its default value — Streamlit's equivalent of frontend component state.
151	<code>"evals_left": 25,</code>	Supplies the <code>evals_left</code> entry (<code>25,`</code>) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
152	<code>"max_evals": 25,</code>	Supplies the <code>max_evals</code> entry (<code>25,`</code>) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
153	<code>"quota_source": "trial",</code>	Supplies the <code>quota_source</code> entry (<code>`"trial,"`</code>) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
154	<code>"current_tier": "Pro Agency Trial",</code>	Supplies the <code>current_tier</code> entry (<code>`"Pro Agency Trial,"`</code>) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
155	<code>"latest_results": None,</code>	Supplies the <code>latest_results</code> entry (<code>None,`</code>) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
156	<code>"pending_plan": None.</code>	Supplies the <code>pending_plan</code> entry (<code>None,`</code>) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.

156	<code> "pending_plan": None,</code>	Supplies the <code>pending_plan</code> entry (<code>None</code> ,) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
157	<code> "pending_amount": None,</code>	Supplies the <code>pending_amount</code> entry (<code>None</code> ,) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
158	<code> "pending_evals": None,</code>	Supplies the <code>pending_evals</code> entry (<code>None</code> ,) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
159	<code> "is_admin": False,</code>	Supplies the <code>is_admin</code> entry (<code>False</code> ,) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
160	<code> "admin_open": False,</code>	Supplies the <code>admin_open</code> entry (<code>False</code> ,) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
161	<code> "show_billing_portal": False,</code>	Supplies the <code>show_billing_portal</code> entry (<code>False</code> ,) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
162	<code> "partial_utr": None,</code>	Supplies the <code>partial_utr</code> entry (<code>None</code> ,) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
163	<code> "analysis_history": [],</code>	Supplies the <code>analysis_history</code> entry (<code>[]</code> ,) as part of the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) started above.
164	<code>}</code>	Closes the multi-line <code>_ss_defaults</code> dictionary (Streamlit session-state defaults) opened above.
165	<code>for k, v in _ss_defaults.items():</code>	Loop — iterates <code>k</code> , <code>v</code> over <code>_ss_defaults.items()</code> , running the indented block once per item.
166	<code> if k not in st.session_state:</code>	Conditional check — runs the indented block below only when <code>k</code> not in <code>st.session_state</code> evaluates to true.
167	<code> st.session_state[k] = v</code>	Inside the current function/block, sets the variable <code>st.session_state[k]</code> to the result of <code>v</code> .
168		Blank line — vertical spacing added purely for readability; has no functional effect.
169	<code>TRIAL_USER_ID = _get_trial_user_id()</code>	At module level, sets the variable <code>TRIAL_USER_ID</code> to the result of <code>_get_trial_user_id()</code> .
170	<code>if "trial_quota_loaded" not in st.session_state:</code>	Conditional check — runs the indented block below only when <code>"trial_quota_loaded"</code> not in <code>st.session_state</code> evaluates to true.
171	<code> st.session_state.eval_left =</code> <code> _trial_remaining(TRIAL_USER_ID)</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>evals_left</code> to <code>_trial_remaining(TRIAL_USER_ID)</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
172	<code> st.session_state.max_evals = TRIAL_LIMIT</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>max_evals</code> to <code>TRIAL_LIMIT</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
173	<code> st.session_state.quota_source = "trial"</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>quota_source</code> to <code>"trial"</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
174	<code> st.session_state.trial_quota_loaded = True</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>trial_quota_loaded</code> to <code>True</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
175		Blank line — vertical spacing added purely for readability; has no functional effect.
176	<code>@st.cache_resource</code>	Decorator: tells Streamlit to cache the return value of the next function across reruns/sessions, so expensive setup (like loading the AI agent) runs only once per app instance.
177	<code>def load_agent():</code>	Defines the function <code>load_agent(...)</code> , which instantiates the <code>DetectiveAgent</code> (the AI/RAG backend) once, then Streamlit caches it.
178	<code> return DetectiveAgent()</code>	Returns <code>DetectiveAgent()</code> as the result of the current function, ending its execution.
179		Blank line — vertical spacing added purely for readability; has no functional effect.
180	<code>agent = load_agent()</code>	At module level, sets the variable <code>agent</code> to the result of <code>load_agent()</code> .
181		Blank line — vertical spacing added purely for readability; has no functional effect.
182	<code>UPI_VPA = "adityasriv@ptyes"</code>	Defines <code>UPI_VPA</code> — the UPI payment address (Virtual Payment Address) customers pay into.
183	<code>UPI_NAME = "Aditya Srivastava"</code>	Defines <code>UPI_NAME</code> — the display name of the payee shown in UPI payment apps.
184		Blank line — vertical spacing added purely for readability; has no functional effect.
185	<code>def make_upi_qr(amount: int, plan_ref: str) -> bytes:</code>	Defines the function <code>make_upi_qr(...)</code> , which builds a scannable UPI payment QR code image for a given amount.
186	<code> upi_uri = (</code>	Inside the current function/block, sets the variable <code>upi_uri</code> to the result of <code>(</code> .
187	<code> f"upi://pay?pa={UPI_VPA}&pn={UPI_NAME.replace(' ',</code> <code> '%20')}"</code>	Continuation of the multi-line <code>upi_uri</code> assignment` above, contributing the text <code>"upi://pay?pa={UPI_VPA}&pn={UPI_NAME.replace(' ', '%20')}"</code> .
188	<code> f"&am={amount}&cu=INR&tn={plan_ref.replace(' ',</code> <code> '_')}"</code>	Continuation of the multi-line <code>upi_uri</code> assignment` above, contributing the text <code>"&am={amount}&cu=INR&tn={plan_ref.replace(' ', '_')}"</code> .
189	<code>)</code>	Closes the multi-line <code>upi_uri</code> assignment` opened above.
190	<code>try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
191	<code> import segno</code>	Imports Python's built-in <code>segno</code> module, used to provide supporting functionality.
192	<code> buf = io.BytesIO()</code>	Inside the current function/block, sets the variable <code>buf</code> to the result of <code>io.BytesIO()</code> .
193	<code> qr = segno.make(upi_uri, error="H")</code>	Inside the current function/block, sets the variable <code>qr</code> to the result of <code>segno.make(upi_uri, error="H")</code> .
194	<code> qr.save(buf, kind="png", scale=8, border=4,</code> <code> dark="black", light="white")</code>	Calls <code>qr.save(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
195	<code> buf.seek(0)</code>	Calls <code>buf.seek(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
196	<code> return buf.getvalue()</code>	Returns <code>buf.getvalue()</code> as the result of the current function, ending its execution.
197	<code>except ImportError:</code>	Catches an error from the <code>try</code> block above of type <code>ImportError</code> , so the app can show a friendly message instead of crashing.
198	<code> pass</code>	No-op placeholder — deliberately does nothing (satisfies Python's block-body requirement).
199		Blank line — vertical spacing added purely for readability; has no functional effect.
200	<code>import qrcode as _qr</code>	Imports <code>qrcode</code> and aliases it as <code>_qr</code> for shorter reference later in the file.
201	<code>import qrcode.constants as _qrc</code>	Imports <code>qrcode.constants</code> and aliases it as <code>_qrc</code> for shorter reference later in the file.
202	<code> q = _qr.QRCode(error_correction=_qrc.ERROR_CORRECT_H,</code> <code> box_size=8, border=4)</code>	Inside the current function/block, sets the variable <code>q</code> to the result of <code>_qr.QRCode(error_correction=_qrc.ERROR_CORRECT_H, box_size=8, border=4)</code> .
203	<code> q.add_data(upi_uri)</code>	Calls <code>q.add_data(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
204	<code> q.make(fit=True)</code>	Calls <code>q.make(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
205	<code> pil_img = q.make_image(fill_color="black",</code> <code> back_color="white").get_image()</code>	Inside the current function/block, sets the variable <code>pil_img</code> to the result of <code>'q.make_image(fill_color="black", back_color="white").get_image()'</code> .
206	<code> buf = io.BytesIO()</code>	Inside the current function/block, sets the variable <code>buf</code> to the result of <code>io.BytesIO()</code> .

206	<code>buf = io.BytesIO()</code>	Inside the current function/block, sets the variable <code>buf</code> to the result of <code>io.BytesIO()</code> .
207	<code> pil_img.save(buf, "PNG")</code>	Calls <code>pil_img.save(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
208	<code> buf.seek(0)</code>	Calls <code>buf.seek(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
209	<code> return buf.getvalue()</code>	Returns <code>buf.getvalue()</code> as the result of the current function, ending its execution.
210		Blank line — vertical spacing added purely for readability; has no functional effect.
211	<code>def generate_pdf_report(suspect_name, age, tendency_score, risk_level, behaviors, matched_cases):</code>	Defines the function <code>generate_pdf_report(...)</code> , which builds a downloadable executive PDF summarising a suspect's profiling results.
212	<code> pdf = FPDF()</code>	Inside the current function/block, sets the variable <code>pdf</code> to the result of <code>FPDF()</code> .
213	<code> pdf.add_page()</code>	Calls <code>pdf.add_page(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
214	<code> pdf.set_font("Helvetica", "B", 16)</code>	Calls <code>pdf.set_font(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
215	<code> pdf.cell(0, 10, "EXECUTIVE CRIMINAL PROFILE REPORT", new_x="LMARGIN", new_y="NEXT", align="C")</code>	Calls <code>pdf.cell(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
216	<code> pdf.ln(5)</code>	Calls <code>pdf.ln(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
217	<code> pdf.set_font("Helvetica", "", 11)</code>	Calls <code>pdf.set_font(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
218	<code> pdf.cell(0, 8, f"Suspect Name: {suspect_name}", new_x="LMARGIN", new_y="NEXT")</code>	Calls <code>pdf.cell(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
219	<code> pdf.cell(0, 8, f"Age: {age if age else 'Unknown'}", new_x="LMARGIN", new_y="NEXT")</code>	Calls <code>pdf.cell(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
220	<code> pdf.cell(0, 8, f"Calculated Tendency Score: {tendency_score}", new_x="LMARGIN", new_y="NEXT")</code>	Calls <code>pdf.cell(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
221	<code> pdf.cell(0, 8, f"Risk Assessment Level: {risk_level}", new_x="LMARGIN", new_y="NEXT")</code>	Calls <code>pdf.cell(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
222	<code> pdf.ln(5)</code>	Calls <code>pdf.ln(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
223	<code> pdf.set_font("Helvetica", "B", 12)</code>	Calls <code>pdf.set_font(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
224	<code> pdf.cell(0, 8, "Observed Behaviors & Modus Operandi:", new_x="LMARGIN", new_y="NEXT")</code>	Calls <code>pdf.cell(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
225	<code> pdf.set_font("Helvetica", "", 10)</code>	Calls <code>pdf.set_font(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
226	<code> pdf.multi_cell(0, 6, str(behaviors))</code>	Calls <code>pdf.multi_cell(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
227	<code> pdf.ln(5)</code>	Calls <code>pdf.ln(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
228	<code> pdf.set_font("Helvetica", "B", 12)</code>	Calls <code>pdf.set_font(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
229	<code> pdf.cell(0, 8, "Matched Historical Precedents:", new_x="LMARGIN", new_y="NEXT")</code>	Calls <code>pdf.cell(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
230	<code> pdf.set_font("Helvetica", "", 10)</code>	Calls <code>pdf.set_font(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
231	<code> for idx, case in enumerate(matched_cases, 1):</code>	Loop — iterates <code>idx, case</code> over <code>enumerate(matched_cases, 1)</code> , running the indented block once per item.
232	<code> pdf.cell(0, 6,</code>	Calls <code>pdf.cell(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
233	<code> f"{idx}. {case.get('case_title','Unknown Case')} ({case.get('location','N/A')})",</code>	Continuation of the multi-line <code>pdf.cell(...)</code> call above, contributing the text “ <code>{idx}. {case.get('case_title','Unknown Case')} ({case...."</code>
234	<code> new_x="LMARGIN", new_y="NEXT")</code>	Continuation of the multi-line <code>pdf.cell(...)</code> call above, contributing the text “ <code>LMARGIN"</code> .
235	<code> snippet = case.get("summary", case.get("snippet", ""))[:150]</code>	Inside the current function/block, sets the variable <code>snippet</code> to the result of <code>`case.get("summary", case.get("snippet", ""))[:150]`</code> .
236	<code> pdf.multi_cell(0, 5, f" Snippet: {snippet}...")</code>	Calls <code>pdf.multi_cell(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
237	<code> pdf.ln(2)</code>	Calls <code>pdf.ln(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
238	<code> return bytes(pdf.output())</code>	Returns <code>bytes(pdf.output())</code> as the result of the current function, ending its execution.
239		Blank line — vertical spacing added purely for readability; has no functional effect.
240	<code># Sidebar</code>	Comment (not executed): “Sidebar” — documents intent for the code that follows.
241	<code>st.sidebar.header("☯ Case Indexer")</code>	Renders a section heading on the page, labelled/showing “☯ Case Indexer”.
242	<code>uploaded_file = st.sidebar.file_uploader("Upload Case JSON to Vector DB", type=["json"])</code>	Renders a file-upload widget on the page, labelled/showing “Upload Case JSON to Vector DB”. The value the user enters/selects is stored in <code>uploaded_file</code> for use later in the script.
243	<code>if uploaded_file is not None:</code>	Conditional check — runs the indented block below only when <code>uploaded_file</code> is not <code>None</code> evaluates to true.
244	<code> try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
245	<code> case_data = json.load(uploaded_file)</code>	Inside the current function/block, sets the variable <code>case_data</code> to the result of <code>json.load(uploaded_file)</code> .
246	<code> cases_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "cases"))</code>	Inside the current function/block, sets the variable <code>cases_dir</code> to the result of <code>`os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "cases"...`</code> .
247	<code> os.makedirs(cases_dir, exist_ok=True)</code>	Calls <code>os.makedirs(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
248	<code> save_path = os.path.join(cases_dir, uploaded_file.name)</code>	Inside the current function/block, sets the variable <code>save_path</code> to the result of <code>os.path.join(cases_dir, uploaded_file.name)</code> .
249	<code> with open(save_path, "w", encoding="utf-8") as f:</code>	Opens a file (<code>`open(save_path, "w", encoding="utf-8") as f`</code>) for reading/writing; the file is automatically closed when the indented block ends.
250	<code> json.dump(case_data, f, indent=4)</code>	Calls <code>json.dump(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
251	<code> indexed_count = agent.reload_cases()</code>	Inside the current function/block, sets the variable <code>indexed_count</code> to the result of <code>agent.reload_cases()</code> .
252	<code> st.sidebar.success(</code>	Renders a green success message box on the page.
253	<code> f"Indexed '{uploaded_file.name}'! The active case index now contains "</code>	Continuation of the multi-line <code>st.sidebar.success(...)</code> call above, contributing the text “ <code>Indexed '{uploaded_file.name}'! The active case index ..."</code> .
254	<code> f"{indexed_count} cases."</code>	Continuation of the multi-line <code>st.sidebar.success(...)</code> call above, contributing the text “ <code>{indexed_count} cases."</code> .
255	<code>)</code>	Closes the multi-line <code>st.sidebar.success(...)</code> call opened above.
256	<code> except Exception as e:</code>	Catches an error from the <code>try</code> block above of type <code>Exception</code> as <code>e</code> , so the app can show a friendly message instead of crashing.
257	<code> st.sidebar.error(f"Upload failed: {e}")</code>	Renders a red error message box on the page, labelled/showing “Upload failed: <code>{e}</code> ”.
258		Blank line — vertical spacing added purely for readability; has no functional effect.

259	<code>st.sidebar.divider()</code>	Renders a horizontal divider line on the page.
260	<code>st.sidebar.markdown("### B2B Agency Plan")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “### B2B Agency Plan”.
261	<code>_max = st.session_state.max_evals</code>	At module level, sets the variable <code>_max</code> to the result of <code>st.session_state.max_evals</code> .
262	<code>_left = st.session_state.evals_left</code>	At module level, sets the variable <code>_left</code> to the result of <code>st.session_state.evals_left</code> .
263	<code>quota_display = "Unlimited" if _max == "Unlimited" else f"{min(_left, _max)}/{_max}"</code>	At module level, sets the variable <code>quota_display</code> to the result of “Unlimited” if <code>_max == "Unlimited"</code> else <code>f"{min(_left, _max)}/{_max}"</code> .
264		Blank line — vertical spacing added purely for readability; has no functional effect.
265	<code>st.sidebar.info()</code>	Renders a blue informational message box on the page.
266	<code> f"""Current Tier:** {st.session_state.current_tier}\n\n"</code>	Continuation of the multi-line <code>st.sidebar.info(...)</code> call above, contributing the text “**Current Tier:** {st.session_state.current_tier}\n\n”.
267	<code> f"""Evaluations Remaining:** {quota_display}"</code>	Continuation of the multi-line <code>st.sidebar.info(...)</code> call above, contributing the text “**Evaluations Remaining:** {quota_display}”.
268	<code>)</code>	Closes the multi-line <code>st.sidebar.info(...)</code> call opened above.
269	<code>if _max != "Unlimited" and isinstance(_left, int) and _left <= 0:</code>	Conditional check — runs the indented block below only when <code>_max != "Unlimited"</code> and <code>isinstance(_left, int)</code> and <code>_left <= 0</code> evaluates to true.
270	<code> st.sidebar.error("⚠ Trial Limit Reached")</code>	Renders a red error message box on the page, labelled/showing “⚠ Trial Limit Reached”.
271		Blank line — vertical spacing added purely for readability; has no functional effect.
272	<code>if st.sidebar.button("🔄 Upgrade / Billing Portal", use_container_width=True, key="sb_upgrade"):</code>	Conditional check — runs the indented block below only when <code>st.sidebar.button("🔄 Upgrade / Billing Portal", use_container_width=True, key="..."</code> evaluates to true.
273	<code> st.session_state.show_billing_portal = not st.session_state.show_billing_portal</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>show_billing_portal</code> to <code>not st.session_state.show_billing_portal</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
274	<code> if not st.session_state.show_billing_portal:</code>	Conditional check — runs the indented block below only when <code>not st.session_state.show_billing_portal</code> evaluates to true.
275	<code> st.session_state.pending_plan = None</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_plan</code> to <code>None</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
276	<code> st.session_state.pending_amount = None</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_amount</code> to <code>None</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
277	<code> st.session_state.pending_evals = None</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_evals</code> to <code>None</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
278	<code>st.rerun()</code>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
279		Blank line — vertical spacing added purely for readability; has no functional effect.
280	<code>if st.session_state.show_billing_portal:</code>	Conditional check — runs the indented block below only when <code>st.session_state.show_billing_portal</code> evaluates to true.
281	<code> PLANS_SIDEBAR = {</code>	Starts a dictionary that defines the subscription plans (name → price / evaluation-count / display label) used to render the pricing cards shown to the user.
282	<code> "🔄 Starter": {"amount": 500, "evals": 100, "label": "₹500/mo — 100 Evals"},</code>	Continuation of the multi-line <code>PLANS_SIDEBAR</code> dictionary (sidebar pricing table) above, contributing the text “🔄 Starter”.
283	<code> "🔄 Pro": {"amount": 1000, "evals": 500, "label": "₹1,000/mo — 500 Evals"},</code>	Continuation of the multi-line <code>PLANS_SIDEBAR</code> dictionary (sidebar pricing table) above, contributing the text “🔄 Pro”.
284	<code> "🔄 Enterprise": {"amount": 2000, "evals": "Unlimited", "label": "₹2,000/mo — Unlimited"},</code>	Continuation of the multi-line <code>PLANS_SIDEBAR</code> dictionary (sidebar pricing table) above, contributing the text “🔄 Enterprise”.
285	<code> }</code>	Closes the multi-line <code>PLANS_SIDEBAR</code> dictionary (sidebar pricing table) opened above.
286	<code>st.sidebar.markdown("#### 🏠 Choose a Plan")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “#### 🏠 Choose a Plan”.
287	<code>for pname, pinfo in PLANS_SIDEBAR.items():</code>	Loop — iterates <code>pname</code> , <code>pinfo</code> over <code>PLANS_SIDEBAR.items()</code> , running the indented block once per item.
288	<code> sb_key = f"sb_plan_{pname.replace('' , '_').replace('/', '_')}"</code>	Inside the current function/block, sets the variable <code>sb_key</code> to the result of <code>f"sb_plan_{pname.replace(' ', '_').replace('/', '_')}"</code> .
289	<code> if st.sidebar.button(f"{pname} — {pinfo['label']}", key=sb_key, use_container_width=True):</code>	Conditional check — runs the indented block below only when <code>st.sidebar.button(f"{pname} — {pinfo['label']}", key=sb_key, use_container_widt...</code> evaluates to true.
290	<code> st.session_state.pending_plan = pname</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_plan</code> to <code>pname</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
291	<code> st.session_state.pending_amount = pinfo["amount"]</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_amount</code> to <code>pinfo["amount"]</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
292	<code> st.session_state.pending_evals = pinfo["evals"]</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_evals</code> to <code>pinfo["evals"]</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
293	<code>st.rerun()</code>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
294		Blank line — vertical spacing added purely for readability; has no functional effect.
295	<code>if st.session_state.pending_plan:</code>	Conditional check — runs the indented block below only when <code>st.session_state.pending_plan</code> evaluates to true.
296	<code> _plan = st.session_state.pending_plan</code>	Inside the current function/block, sets the variable <code>_plan</code> to the result of <code>st.session_state.pending_plan</code> .
297	<code> _amount = st.session_state.pending_amount</code>	Inside the current function/block, sets the variable <code>_amount</code> to the result of <code>st.session_state.pending_amount</code> .
298	<code> _evals = st.session_state.pending_evals</code>	Inside the current function/block, sets the variable <code>_evals</code> to the result of <code>st.session_state.pending_evals</code> .
299	<code>st.sidebar.markdown(f"---\n**🏠 Pay for {_plan}**")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “---\n**🏠 Pay for {_plan}**”.
300	<code>st.sidebar.markdown(f"Amount: **₹{_amount:,}** · UPI: `{UPI_VPA}`")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “Amount: **₹{_amount:,}** · UPI: <code>{UPI_VPA}</code> ”.
301	<code>try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
302	<code> _qr_bytes = make_upi_qr(_amount, f"Plan_Upgrade_{_plan.split()[-1]}")</code>	Inside the current function/block, sets the variable <code>_qr_bytes</code> to the result of <code>make_upi_qr(_amount, f"Plan_Upgrade_{_plan.split()[-1]}")</code> .
303	<code>st.sidebar.image(_qr_bytes, caption=f"Scan to Pay ₹{_amount:,}", width=200)</code>	Renders an image element on the page, labelled/showing “Scan to Pay ₹{_amount:,}”.
304	<code>except Exception:</code>	Catches an error from the <code>try</code> block above of type <code>Exception</code> , so the app can show a friendly message instead of crashing.
305	<code>st.sidebar.code(</code>	Renders a monospaced code block on the page.
306	<code> f"upi://pay?pa=</code>	Continuation of the multi-line <code>st.sidebar.code(...)</code> call above, contributing the text <code>upi://pay?pa=</code>

<pre>{UPI_VPA}&pn=Aditya%20Srivastava"</pre>	<pre>{UPI_VPA}&pn=Aditya%20Srivastava".</pre>
<pre>307 f"&am={_amount}&cu=INR&tn=Plan_Upgrade",</pre>	Continuation of the multi-line <code>st.sidebar.code(...)</code> call above, contributing the text <code>"&am={_amount}&cu=INR&tn=Plan_Upgrade"</code> .
<pre>308 language="text"</pre>	Continuation of the multi-line <code>st.sidebar.code(...)</code> call above, contributing the text <code>"text"</code> .
<pre>309)</pre>	Closes the multi-line <code>st.sidebar.code(...)</code> call opened above.
<pre>310 with st.sidebar.form(key="sb_utr_form"):</pre>	Opens a context that places every widget defined in the indented block into the left-hand sidebar.
<pre>311 _utr = st.text_input("UTR / Transaction Ref No.", placeholder="e.g. 426789012345", max_chars=30)</pre>	Renders a single-line text input box on the page, labelled/showing <code>"UTR / Transaction Ref No."</code> . The value the user enters/selects is stored in <code>_utr</code> for use later in the script.
<pre>312 _paid_str = st.text_input("Amount You Paid (₹)", placeholder=f"e.g. {_amount}", max_chars=10)</pre>	Renders a single-line text input box on the page, labelled/showing <code>"Amount You Paid (₹)"</code> . The value the user enters/selects is stored in <code>_paid_str</code> for use later in the script.
<pre>313 _sub = st.form_submit_button("👉 Submit Payment Proof", use_container_width=True)</pre>	Renders a submit button bound to the enclosing form on the page, labelled/showing <code>"👉 Submit Payment Proof"</code> . The value the user enters/selects is stored in <code>_sub</code> for use later in the script.
<pre>314 if _sub:</pre>	Conditional check — runs the indented block below only when <code>_sub</code> evaluates to true.
<pre>315 if not _utr.strip():</pre>	Conditional check — runs the indented block below only when <code>not _utr.strip()</code> evaluates to true.
<pre>316 st.sidebar.error("Please enter your UTR number.")</pre>	Renders a red error message box on the page, labelled/showing <code>"Please enter your UTR number."</code> .
<pre>317 elif not _paid_str.strip():</pre>	Conditional check — runs the indented block below only when <code>not _paid_str.strip()</code> evaluates to true.
<pre>318 st.sidebar.error("Please enter the amount you paid.")</pre>	Renders a red error message box on the page, labelled/showing <code>"Please enter the amount you paid."</code> .
<pre>319 else:</pre>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
<pre>320 try:</pre>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
<pre>321 _paid_val = float(_paid_str.replace(",", "").strip())</pre>	Inside the current function/block, sets the variable <code>_paid_val</code> to the result of <code>`float(_paid_str.replace(",", "").strip())`</code> .
<pre>322 except ValueError:</pre>	Catches an error from the <code>try</code> block above of type <code>ValueError</code> , so the app can show a friendly message instead of crashing.
<pre>323 st.sidebar.error("Invalid amount — enter a number.")</pre>	Renders a red error message box on the page, labelled/showing <code>"Invalid amount — enter a number."</code> .
<pre>324 _paid_val = None</pre>	Inside the current function/block, sets the variable <code>_paid_val</code> to the result of <code>None</code> .
<pre>325 if _paid_val is not None:</pre>	Conditional check — runs the indented block below only when <code>_paid_val</code> is not <code>None</code> evaluates to true.
<pre>326 _status, _msg, _remaining = submit_payment(</pre>	Inside the current function/block, sets the variable <code>_status, _msg, _remaining</code> to the result of <code>submit_payment(</code> .
<pre>327 plan=plan, required_amount=float(_amount),</pre>	Continuation line of the multi-line <code>`_status, _msg, _remaining`</code> assignment` statement started above.
<pre>328 amount_paid=_paid_val, evals=_evals, utr=_utr.strip())</pre>	Continuation line of the multi-line <code>`_status, _msg, _remaining`</code> assignment` statement started above.
<pre>329)</pre>	Closes the multi-line <code>`_status, _msg, _remaining`</code> assignment` opened above.
<pre>330 if _status == STATUS_FLAGGED:</pre>	Conditional check — runs the indented block below only when <code>_status == STATUS_FLAGGED</code> evaluates to true.
<pre>331 st.sidebar.error(_msg)</pre>	Renders a red error message box on the page.
<pre>332 elif _status == STATUS_PARTIAL:</pre>	Conditional check — runs the indented block below only when <code>_status == STATUS_PARTIAL</code> evaluates to true.
<pre>333 st.sidebar.warning(_msg)</pre>	Renders an amber warning message box on the page.
<pre>334 st.session_state.partial_utr = _utr.strip()</pre>	Updates Streamlit's persistent <code>session_state</code> — sets <code>partial_utr</code> to <code>_utr.strip()</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
<pre>335 st.session_state.show_billing_portal = False</pre>	Updates Streamlit's persistent <code>session_state</code> — sets <code>show_billing_portal</code> to <code>False</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
<pre>336 st.rerun()</pre>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
<pre>337 else:</pre>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
<pre>338 st.sidebar.success("✓ Submitted! Quota will be unlocked after admin verification.")</pre>	Renders a green success message box on the page, labelled/showing <code>"✓ Submitted! Quota will be unlocked after admin verificatio..."</code> .
<pre>339 st.session_state.pending_plan = None</pre>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_plan</code> to <code>None</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
<pre>340 st.session_state.pending_amount = None</pre>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_amount</code> to <code>None</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
<pre>341 st.session_state.pending_evals = None</pre>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_evals</code> to <code>None</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
<pre>342 st.session_state.show_billing_portal = False</pre>	Updates Streamlit's persistent <code>session_state</code> — sets <code>show_billing_portal</code> to <code>False</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
<pre>343 st.rerun()</pre>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
<pre>344</pre>	Blank line — vertical spacing added purely for readability; has no functional effect.
<pre>345 if st.session_state.is_admin:</pre>	Conditional check — runs the indented block below only when <code>st.session_state.is_admin</code> evaluates to true.
<pre>346 if st.sidebar.button("🔄 Reset Demo & Clear Cache", use_container_width=True, key="sb_reset"):</pre>	Conditional check — runs the indented block below only when <code>`st.sidebar.button("🔄 Reset Demo & Clear Cache", use_container_width=True, key="..."`</code> evaluates to true.
<pre>347 for k, v in _ss_defaults.items():</pre>	Loop — iterates <code>k, v</code> over <code>_ss_defaults.items()</code> , running the indented block once per item.
<pre>348 st.session_state[k] = v</pre>	Inside the current function/block, sets the variable <code>st.session_state[k]</code> to the result of <code>v</code> .
<pre>349 st.session_state.evals_left = _trial_remaining(TRIAL_USER_ID)</pre>	Updates Streamlit's persistent <code>session_state</code> — sets <code>evals_left</code> to <code>_trial_remaining(TRIAL_USER_ID)</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
<pre>350 st.session_state.max_evals = TRIAL_LIMIT</pre>	Updates Streamlit's persistent <code>session_state</code> — sets <code>max_evals</code> to <code>TRIAL_LIMIT</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
<pre>351 st.session_state.quota_source = "trial"</pre>	Updates Streamlit's persistent <code>session_state</code> — sets <code>quota_source</code> to <code>"trial"</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
<pre>352 st.session_state.trial_quota_loaded = True</pre>	Updates Streamlit's persistent <code>session_state</code> — sets <code>trial_quota_loaded</code> to <code>True</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
<pre>353 st.rerun()</pre>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how

353		forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
354		Blank line — vertical spacing added purely for readability; has no functional effect.
355	<code>st.sidebar.divider()</code>	Renders a horizontal divider line on the page.
356	<code>with st.sidebar.expander("🏠 Admin Portal", expanded=False):</code>	Opens a context that places every widget defined in the indented block into the left-hand sidebar.
357	<code>if st.session_state.is_admin:</code>	Conditional check — runs the indented block below only when <code>st.session_state.is_admin</code> evaluates to true.
358	<code>st.success("✓ Logged in as Admin")</code>	Renders a green success message box on the page, labelled/showing "✓ Logged in as Admin".
359	<code>if st.button("🚪 Logout Admin", use_container_width=True, key="btn_admin_logout"):</code>	Conditional check — runs the indented block below only when <code>st.button("🚪 Logout Admin", use_container_width=True, key="btn_admin_logout")</code> evaluates to true.
360	<code>st.session_state.is_admin = False</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>is_admin</code> to <code>False</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
361	<code>st.session_state.admin_open = False</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>admin_open</code> to <code>False</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
362	<code>st.session_state.evals_left = _trial_remaining(TRIAL_USER_ID)</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>evals_left</code> to <code>_trial_remaining(TRIAL_USER_ID)</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
363	<code>st.session_state.max_evals = TRIAL_LIMIT</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>max_evals</code> to <code>TRIAL_LIMIT</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
364	<code>st.session_state.quota_source = "trial"</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>quota_source</code> to <code>"trial"</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
365	<code>st.rerun()</code>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
366	<code>else:</code>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
367	<code>admin_pw = st.text_input("Admin 3-digit PIN", type="password", placeholder="Enter 3-digit admin PIN...", max_chars=3, key="admin_pw_input")</code>	Renders a single-line text input box on the page, labelled/showing "Admin 3-digit PIN". The value the user enters/selects is stored in <code>admin_pw</code> for use later in the script.
368	<code>if st.button("🔑 Login", use_container_width=True, key="btn_admin_login"):</code>	Conditional check — runs the indented block below only when <code>st.button("🔑 Login", use_container_width=True, key="btn_admin_login")</code> evaluates to true.
369	<code>if re.fullmatch(r"\d{3}", admin_pw) and admin_pw == _get_admin_pin():</code>	Conditional check — runs the indented block below only when <code>re.fullmatch(r"\d{3}", admin_pw)</code> and <code>admin_pw == _get_admin_pin()</code> evaluates to true.
370	<code>st.session_state.is_admin = True</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>is_admin</code> to <code>True</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
371	<code>st.session_state.evals_left = "Unlimited"</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>evals_left</code> to <code>"Unlimited"</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
372	<code>st.session_state.max_evals = "Unlimited"</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>max_evals</code> to <code>"Unlimited"</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
373	<code>st.session_state.quota_source = "admin"</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>quota_source</code> to <code>"admin"</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
374	<code>st.rerun()</code>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
375	<code>else:</code>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
376	<code>st.error("✗ Invalid or incorrect 3-digit admin PIN.")</code>	Renders a red error message box on the page, labelled/showing "✗ Invalid or incorrect 3-digit admin PIN.".
377		Blank line — vertical spacing added purely for readability; has no functional effect.
378	<code>if st.session_state.is_admin:</code>	Conditional check — runs the indented block below only when <code>st.session_state.is_admin</code> evaluates to true.
379	<code>st.sidebar.divider()</code>	Renders a horizontal divider line on the page.
380	<code>if st.sidebar.button("💸 Admin Payment Approvals", use_container_width=True, key="sb_admin"):</code>	Conditional check — runs the indented block below only when <code>st.sidebar.button("💸 Admin Payment Approvals", use_container_width=True, key="sb_admin")</code> evaluates to true.
381	<code>st.session_state.admin_open = not st.session_state.admin_open</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>admin_open</code> to <code>not st.session_state.admin_open</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
382	<code>st.rerun()</code>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
383		Blank line — vertical spacing added purely for readability; has no functional effect.
384	<code>if st.session_state.admin_open:</code>	Conditional check — runs the indented block below only when <code>st.session_state.admin_open</code> evaluates to true.
385	<code>st.sidebar.markdown("#### 📄 All Pending Submissions")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing "#### 📄 All Pending Submissions".
386	<code>st.sidebar.warning()</code>	Renders an amber warning message box on the page.
387	<code>"UTRs and amounts are user claims until checked in your UPI/bank app. "</code>	Continuation of the multi-line <code>st.sidebar.warning(...)</code> call above, contributing the text "UTRs and amounts are user claims until checked in your...".
388	<code>"Approve only after the transaction, amount, and payee match."</code>	Continuation of the multi-line <code>st.sidebar.warning(...)</code> call above, contributing the text "Approve only after the transaction, amount, and payee ...".
389	<code>)</code>	Closes the multi-line <code>st.sidebar.warning(...)</code> call opened above.
390	<code>_all_pending = get_pending_payments()</code>	Inside the current function/block, sets the variable <code>_all_pending</code> to the result of <code>get_pending_payments()</code> .
391	<code>if not _all_pending:</code>	Conditional check — runs the indented block below only when <code>not _all_pending</code> evaluates to true.
392	<code>st.sidebar.info("No pending submissions.")</code>	Renders a blue informational message box on the page, labelled/showing "No pending submissions.".
393	<code>else:</code>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
394	<code>for i, pmt in enumerate(_all_pending):</code>	Loop — iterates <code>i, pmt</code> over <code>enumerate(_all_pending)</code> , running the indented block once per item.
395	<code>_putr = pmt.get("utr", "")</code>	Inside the current function/block, sets the variable <code>_putr</code> to the result of <code>pmt.get("utr", "")</code> .
396	<code>_preq = pmt.get("required_amount", pmt.get("amount", 0))</code>	Inside the current function/block, sets the variable <code>_preq</code> to the result of <code>pmt.get("required_amount", pmt.get("amount", 0))</code> .
397	<code>_ppaid = pmt.get("amount_paid", pmt.get("amount", 0))</code>	Inside the current function/block, sets the variable <code>_ppaid</code> to the result of <code>pmt.get("amount_paid", pmt.get("amount", 0))</code> .
398	<code>_premain = pmt.get("remaining_balance", 0)</code>	Inside the current function/block, sets the variable <code>_premain</code> to the result of <code>pmt.get("remaining_balance", 0)</code> .
399	<code>_pstatus = pmt.get("status", "")</code>	Inside the current function/block, sets the variable <code>_pstatus</code> to the result of <code>pmt.get("status", "")</code> .
400	<code>_pevals = pmt.get("evals", "?")</code>	Inside the current function/block, sets the variable <code>_pevals</code> to the result of <code>pmt.get("evals", "?")</code> .

401	<code>_pplan = pmt.get("plan", "?")</code>	Inside the current function/block, sets the variable <code>_pplan</code> to the result of <code>`pmt.get("plan", "?")`</code> .
402	<code>_label = f"#{i+1} {_pplan} — UTR: {_putr}"</code>	Inside the current function/block, sets the variable <code>_label</code> to the result of <code>`f"#{i+1} {_pplan} — UTR: {_putr}"`</code> .
403	<code>with st.sidebar.expander(_label):</code>	Opens a context that places every widget defined in the indented block into the left-hand sidebar.
404	<code>st.write(f"***Plan:** {_pplan}")</code>	Renders generic auto-formatted output text on the page, labelled/showing <code>***Plan:** {_pplan}</code> .
405	<code>st.write(f"***Required:** ₹{_preq:,}")</code>	Renders generic auto-formatted output text on the page, labelled/showing <code>***Required:** ₹{_preq:,}</code> .
406	<code>st.write(f"***Paid:** ₹{_ppaid:,}")</code>	Renders generic auto-formatted output text on the page, labelled/showing <code>***Paid:** ₹{_ppaid:,}</code> .
407	<code>if _premain:</code>	Conditional check — runs the indented block below only when <code>_premain</code> evaluates to true.
408	<code>st.write(f"***Deficit:** ₹{_premain:,}")</code>	Renders generic auto-formatted output text on the page, labelled/showing <code>***Deficit:** ₹{_premain:,}</code> .
409	<code>st.write(f"***Status:** {_pstatus}")</code>	Renders generic auto-formatted output text on the page, labelled/showing <code>***Status:** {_pstatus}</code> .
410	<code>st.write(f"***UTR:** ` {_putr}`")</code>	Renders generic auto-formatted output text on the page, labelled/showing <code>***UTR:** {_putr}</code> .
411	<code>st.write(f"***Submitted:** {pmt.get('timestamp', '')}")</code>	Renders generic auto-formatted output text on the page, labelled/showing <code>***Submitted:** {pmt.get('timestamp', '')}</code> .
412	<code>if pmt.get("topup_utrs"):</code>	Conditional check — runs the indented block below only when <code>`pmt.get("topup_utrs")`</code> evaluates to true.
413	<code>st.write(f"***Top-up UTRs:** {'', '.join(pmt['topup_utrs'])}")</code>	Renders generic auto-formatted output text on the page, labelled/showing <code>***Top-up UTRs:** {'', '.join(pmt['topup_utrs'])}</code> .
414	<code>_acol, _fcol = st.columns(2)</code>	Splits the page horizontally into side-by-side columns, letting several pieces of content (e.g. a QR code and payment instructions) sit next to each other.
415	<code>if _acol.button("✓ Approve After Bank Check", key=f"pay_verify_{_putr}_approve"):</code>	Conditional check — runs the indented block below only when <code>`_acol.button("✓ Approve After Bank Check", key=f"pay_verify_{_putr}_approve")`</code> evaluates to true.
416	<code>if approve_payment(_putr):</code>	Conditional check — runs the indented block below only when <code>approve_payment(_putr)</code> evaluates to true.
417	<code>evals = _pevals</code>	Inside the current function/block, sets the variable <code>evals</code> to the result of <code>_pevals</code> .
418	<code>if evals == "Unlimited":</code>	Conditional check — runs the indented block below only when <code>evals == "Unlimited"</code> evaluates to true.
419	<code>st.session_state.eval_left = "Unlimited"</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>evals_left</code> to <code>"Unlimited"</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
420	<code>st.session_state.max_evals = "Unlimited"</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>max_evals</code> to <code>"Unlimited"</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
421	<code>st.session_state.quota_source = "paid"</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>quota_source</code> to <code>"paid"</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
422	<code>else:</code>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
423	<code>try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
424	<code>st.session_state.eval_left = int(evals)</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>evals_left</code> to <code>int(evals)</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
425	<code>st.session_state.max_evals = int(evals)</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>max_evals</code> to <code>int(evals)</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
426	<code>st.session_state.quota_source = "paid"</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>quota_source</code> to <code>"paid"</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
427	<code>except Exception:</code>	Catches an error from the <code>try</code> block above of type <code>Exception</code> , so the app can show a friendly message instead of crashing.
428	<code>pass</code>	No-op placeholder — deliberately does nothing (satisfies Python's block-body requirement).
429	<code>st.session_state.current_tier = _pplan</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>current_tier</code> to <code>_pplan</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
430	<code>st.sidebar.success(f"✓ Quota unlocked — {_putr}")</code>	Renders a green success message box on the page, labelled/showing <code>✓ Quota unlocked — {_putr}</code> .
431	<code>st.rerun()</code>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
432	<code>if _fcol.button("⚠ Flag Partial", key=f"pay_verify_{_putr}_flag"):</code>	Conditional check — runs the indented block below only when <code>`_fcol.button("⚠ Flag Partial", key=f"pay_verify_{_putr}_flag")`</code> evaluates to true.
433	<code>if flag_partial(_putr):</code>	Conditional check — runs the indented block below only when <code>flag_partial(_putr)</code> evaluates to true.
434	<code>st.session_state.partial utr = _putr</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>partial utr</code> to <code>_putr</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
435	<code>st.sidebar.warning("Flagged as partial — user will be prompted for top-up.")</code>	Renders an amber warning message box on the page, labelled/showing <code>Flagged as partial — user will be prompted for top-up.</code> .
436	<code>st.rerun()</code>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
437		Blank line — vertical spacing added purely for readability; has no functional effect.
438	<code>st.sidebar.divider()</code>	Renders a horizontal divider line on the page.
439	<code>st.sidebar.markdown(</code>	Renders formatted Markdown/HTML text output on the page.
440	<code>***📧 Feedback & Suggestions**\n\n"</code>	Continuation of the multi-line <code>st.sidebar.markdown(...)</code> call above, contributing the text <code>***📧 Feedback & Suggestions**\n\n"</code> .
441	<code>"Found a bug? Want a new feature?\n\n"</code>	Continuation of the multi-line <code>st.sidebar.markdown(...)</code> call above, contributing the text <code>"Found a bug? Want a new feature?\n\n"</code> .
442	<code>📧 [yeahboyadi@gmail.com] (mailto:yeahboyadi@gmail.com)\n"</code>	Continuation of the multi-line <code>st.sidebar.markdown(...)</code> call above, contributing the text <code>📧 [yeahboyadi@gmail.com] (mailto:yeahboyadi@gmail.com) ..."</code> .
443	<code>📧 [akshat.v216@gmail.com] (mailto:akshat.v216@gmail.com)"</code>	Continuation of the multi-line <code>st.sidebar.markdown(...)</code> call above, contributing the text <code>📧 [akshat.v216@gmail.com] (mailto:akshat.v216@gmail.com)..."</code> .
444	<code>)</code>	Closes the multi-line <code>st.sidebar.markdown(...)</code> call opened above.
445		Blank line — vertical spacing added purely for readability; has no functional effect.
446	<code>st.title("🕵️ Detective AI")</code>	Renders the page's main title heading on the page, labelled/showing <code>🕵️ Detective AI</code> .
447	<code>st.markdown("Detective Agentic AI & RAG Profiling System : Automated criminal pattern recognition, risk evaluation, and precedent retrieval engine.")</code>	Renders formatted Markdown/HTML text output on the page.
448	<code>st.divider()</code>	Renders a horizontal divider line on the page.
449		Blank line — vertical spacing added purely for readability; has no functional effect.

450	<code>if st.session_state.partial_utr:</code>	Conditional check — runs the indented block below only when <code>st.session_state.partial_utr</code> evaluates to true.
451	<code> _prec = get_partial_by_utr(st.session_state.partial_utr)</code>	Inside the current function/block, sets the variable <code>_prec</code> to the result of <code>get_partial_by_utr(st.session_state.partial_utr)</code> .
452	<code> if _prec:</code>	Conditional check — runs the indented block below only when <code>_prec</code> evaluates to true.
453	<code> _p_paid = _prec.get("amount_paid", 0)</code>	Inside the current function/block, sets the variable <code>_p_paid</code> to the result of <code>_prec.get("amount_paid", 0)</code> .
454	<code> _p_req = _prec.get("required_amount", 0)</code>	Inside the current function/block, sets the variable <code>_p_req</code> to the result of <code>_prec.get("required_amount", 0)</code> .
455	<code> _p_remain = _prec.get("remaining_balance", 0)</code>	Inside the current function/block, sets the variable <code>_p_remain</code> to the result of <code>_prec.get("remaining_balance", 0)</code> .
456	<code> _p_plan = _prec.get("plan", "")</code>	Inside the current function/block, sets the variable <code>_p_plan</code> to the result of <code>_prec.get("plan", "")</code> .
457	<code> _p_utr = _prec.get("utr", "")</code>	Inside the current function/block, sets the variable <code>_p_utr</code> to the result of <code>_prec.get("utr", "")</code> .
458	<code> st.warning(</code>	Renders an amber warning message box on the page.
459	<code> f"⚠️ **Payment Incomplete** — You paid ₹ {_p_paid:,.0f} out of "</code>	Continuation of the multi-line <code>st.warning(...)</code> call above, contributing the text “⚠️ **Payment Incomplete** — You paid ₹ {_p_paid:,.0f} o...”.
460	<code> f"₹{_p_req:,.0f} for the **{_p_plan}** plan. "</code>	Continuation of the multi-line <code>st.warning(...)</code> call above, contributing the text “₹{_p_req:,.0f} for the **{_p_plan}** plan.”.
461	<code> f"Please pay the remaining balance of **₹ {_p_remain:,.0f}** to unlock your evaluations."</code>	Continuation of the multi-line <code>st.warning(...)</code> call above, contributing the text “Please pay the remaining balance of **₹ {_p_remain:,.0f}** ...”.
462	<code>)</code>	Closes the multi-line <code>st.warning(...)</code> call opened above.
463	<code> _tp_qr_col, _tp_inst_col = st.columns([1, 2])</code>	Splits the page horizontally into side-by-side columns, letting several pieces of content (e.g. a QR code and payment instructions) sit next to each other.
464	<code> with _tp_qr_col:</code>	Enters one of the layout columns created above (<code>_tp_qr_col</code>) — every widget in the indented block is rendered inside that column.
465	<code> try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
466	<code> _tp_qr = make_upi_qr(int(_p_remain), f"TopUp_{_p_plan.split()[0]}")</code>	Inside the current function/block, sets the variable <code>_tp_qr</code> to the result of <code>make_upi_qr(int(_p_remain), f"TopUp_{_p_plan.split()[0]}")</code> .
467	<code> st.image(_tp_qr, caption=f"Scan to Pay ₹ {_p_remain:,.0f} (balance)", width=200)</code>	Renders an image element on the page, labelled/showing “Scan to Pay ₹{_p_remain:,.0f} (balance)”.
468	<code> except Exception:</code>	Catches an error from the <code>try</code> block above of type <code>Exception</code> , so the app can show a friendly message instead of crashing.
469	<code> st.code(</code>	Renders a monospaced code block on the page.
470	<code> f"upi://pay?pa= {UPI_VPA}&pn=Aditya%20Srivastava"</code>	Continuation of the multi-line <code>st.code(...)</code> call above, contributing the text “upi://pay?pa= {UPI_VPA}&pn=Aditya%20Srivastava”.
471	<code> f"&am= {int(_p_remain)}&cu=INR&tn=TopUp_Balance",</code>	Continuation of the multi-line <code>st.code(...)</code> call above, contributing the text “&am= {int(_p_remain)}&cu=INR&tn=TopUp_Balance”.
472	<code> language="text"</code>	Continuation of the multi-line <code>st.code(...)</code> call above, contributing the text “text”.
473	<code>)</code>	Closes the multi-line <code>st.code(...)</code> call opened above.
474	<code> with _tp_inst_col:</code>	Enters one of the layout columns created above (<code>_tp_inst_col</code>) — every widget in the indented block is rendered inside that column.
475	<code> st.markdown(</code>	Renders formatted Markdown/HTML text output on the page.
476	<code> f"""UPI ID:** `{UPI_VPA}` \n"</code>	Continuation of the multi-line <code>st.markdown(...)</code> call above, contributing the text “**UPI ID:** {UPI_VPA} \n”.
477	<code> f"""Amount:** ₹{_p_remain:,.0f} \n"</code>	Continuation of the multi-line <code>st.markdown(...)</code> call above, contributing the text “**Amount:** ₹{_p_remain:,.0f} \n”.
478	<code> f"""Payee:** Aditya Srivastava"</code>	Continuation of the multi-line <code>st.markdown(...)</code> call above, contributing the text “**Payee:** Aditya Srivastava”.
479	<code>)</code>	Closes the multi-line <code>st.markdown(...)</code> call opened above.
480	<code> with st.form(key=f"topup_form_{_p_utr}"): </code>	Opens a Streamlit form container — inputs placed inside it are batched together and only submitted when its submit button is pressed.
481	<code> _topup_utr = st.text_input("Enter Top-Up UTR / Transaction Ref", placeholder="New 12-digit UTR after paying balance", max_chars=30)</code>	Renders a single-line text input box on the page, labelled/showing “Enter Top-Up UTR / Transaction Ref”. The value the user enters/selects is stored in <code>_topup_utr</code> for use later in the script.
482	<code> _topup_sub = st.form_submit_button("👤 Submit Top-Up Proof", use_container_width=True, type="primary")</code>	Renders a submit button bound to the enclosing form on the page, labelled/showing “👤 Submit Top-Up Proof”. The value the user enters/selects is stored in <code>_topup_sub</code> for use later in the script.
483	<code> if _topup_sub:</code>	Conditional check — runs the indented block below only when <code>_topup_sub</code> evaluates to true.
484	<code> if not _topup_utr.strip():</code>	Conditional check — runs the indented block below only when <code>not _topup_utr.strip()</code> evaluates to true.
485	<code> st.error("Please enter your top-up UTR.")</code>	Renders a red error message box on the page, labelled/showing “Please enter your top-up UTR.”.
486	<code> else:</code>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
487	<code> _ts, _tm, _tr = submit_topup(</code>	Inside the current function/block, sets the variable <code>_ts, _tm, _tr</code> to the result of <code>submit_topup(</code> .
488	<code> _topup_utr.strip(), _p_utr, float(_p_remain)</code>	Continuation line of the multi-line <code>_ts, _tm, _tr</code> assignment` statement started above.
489	<code>)</code>	Closes the multi-line <code>_ts, _tm, _tr</code> assignment` opened above.
490	<code> if "⚡" in _tm:</code>	Conditional check — runs the indented block below only when <code>"⚡" in _tm`</code> evaluates to true.
491	<code> st.success(_tm)</code>	Renders a green success message box on the page.
492	<code> st.session_state.partial_utr = None</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>partial_utr</code> to <code>None</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
493	<code> st.rerun()</code>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
494	<code> else:</code>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
495	<code> st.error(_tm)</code>	Renders a red error message box on the page.
496	<code> st.divider()</code>	Renders a horizontal divider line on the page.
497	<code> else:</code>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
498	<code> st.session_state.partial_utr = None</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>partial_utr</code> to <code>None</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
499		Blank line — vertical spacing added purely for readability; has no functional effect.

500	<code>_tab_labels = ["🔍 Profiling Dashboard", "💰 Billing & Plans", "📧 Contact & Feedback"]</code>	At module level, sets the variable <code>_tab_labels</code> to the result of <code>["🔍 Profiling Dashboard", "💰 Billing & Plans", "📧 Contact & Feedback"]</code> .
501	<code>if st.session_state.is_admin:</code>	Conditional check — runs the indented block below only when <code>st.session_state.is_admin</code> evaluates to true.
502	<code> _tab_labels.append("🔗 B2B Agency Acquisition")</code>	Calls <code>_tab_labels.append(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
503	<code>_tabs = st.tabs(_tab_labels)</code>	Renders a set of tabbed panels on the page. The individual column/tab objects are stored in <code>_tabs</code> for use in the <code>with</code> blocks below.
504	<code>tab_profile = _tabs[0]</code>	At module level, sets the variable <code>tab_profile</code> to the result of <code>_tabs[0]</code> .
505	<code>tab_billing = _tabs[1]</code>	At module level, sets the variable <code>tab_billing</code> to the result of <code>_tabs[1]</code> .
506	<code>tab_contact = _tabs[2]</code>	At module level, sets the variable <code>tab_contact</code> to the result of <code>_tabs[2]</code> .
507	<code>tab_outreach = _tabs[3] if st.session_state.is_admin else None</code>	At module level, sets the variable <code>tab_outreach</code> to the result of <code>_tabs[3] if st.session_state.is_admin else None</code> .
508		Blank line — vertical spacing added purely for readability; has no functional effect.
509	<code>with tab_profile:</code>	Enters the <code>tab_profile</code> tab panel — everything indented below is rendered only while that tab is active.
510	<code> col1, col2 = st.columns([1, 1])</code>	Splits the page horizontally into side-by-side columns, letting several pieces of content (e.g. a QR code and payment instructions) sit next to each other.
511	<code> with col1:</code>	Enters one of the layout columns created above (<code>col1</code>) — every widget in the indented block is rendered inside that column.
512	<code> st.subheader("Suspect Information & Observations")</code>	Renders a secondary section heading on the page, labelled/showing “Suspect Information & Observations”.
513	<code> with st.form(key="suspect_profiling_form"):</code>	Opens a Streamlit form container — inputs placed inside it are batched together and only submitted when its submit button is pressed.
514	<code> suspect_name = st.text_input("Suspect Name / Alias", placeholder="John Doe")</code>	Renders a single-line text input box on the page, labelled/showing “Suspect Name / Alias”. The value the user enters/selects is stored in <code>suspect_name</code> for use later in the script.
515	<code> age = st.text_input("Age", placeholder="34")</code>	Renders a single-line text input box on the page, labelled/showing “Age”. The value the user enters/selects is stored in <code>age</code> for use later in the script.
516	<code> behaviors = st.text_area("Observed Behaviors, MO, & Traits", height=180,</code>	Renders a multi-line text box on the page, labelled/showing “Observed Behaviors, MO, & Traits”. The value the user enters/selects is stored in <code>behaviors</code> for use later in the script.
517	<code> placeholder="Entering residential premises during late hours, targeting locked cabinets...")</code>	Continuation of the multi-line <code>st.text_area(...)</code> call above, contributing the text “Entering residential premises during late hours, target...”.
518	<code> submit_btn = st.form_submit_button("Run Intelligence Analysis", type="primary", use_container_width=True)</code>	Renders a submit button bound to the enclosing form on the page, labelled/showing “Run Intelligence Analysis”. The value the user enters/selects is stored in <code>submit_btn</code> for use later in the script.
519	<code> with col2:</code>	Enters one of the layout columns created above (<code>col2</code>) — every widget in the indented block is rendered inside that column.
520	<code> st.subheader("Analysis & Precedent Results")</code>	Renders a secondary section heading on the page, labelled/showing “Analysis & Precedent Results”.
521	<code> if submit_btn:</code>	Conditional check — runs the indented block below only when <code>submit_btn</code> evaluates to true.
522	<code> if not behaviors.strip():</code>	Conditional check — runs the indented block below only when <code>not behaviors.strip()</code> evaluates to true.
523	<code> st.warning("Please enter observed behaviors to analyze.")</code>	Renders an amber warning message box on the page, labelled/showing “Please enter observed behaviors to analyze.”.
524	<code> elif st.session_state.max_evals != "Unlimited" and st.session_state.evals_left <= 0:</code>	Conditional check — runs the indented block below only when <code>st.session_state.max_evals != "Unlimited"</code> and <code>st.session_state.evals_left <= 0</code> evaluates to true.
525	<code> st.error("🚫 Evaluation Quota Exceeded! Please upgrade via the Billing tab.")</code>	Renders a red error message box on the page, labelled/showing “🚫 Evaluation Quota Exceeded! Please upgrade via the Billing...”.
526	<code> else:</code>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
527	<code> with st.spinner("Analyzing traits against ChromaDB precedent vectors..."):</code>	Shows a loading spinner (“Analyzing traits against ChromaDB precedent vecto...”) while the indented block runs, giving the user visual feedback that work is in progress.
528	<code> try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
529	<code> name_str = suspect_name.strip() if suspect_name.strip() else "Unnamed Suspect"</code>	Inside the current function/block, sets the variable <code>name_str</code> to the result of <code>suspect_name.strip()</code> if <code>suspect_name.strip()</code> else “Unnamed Suspect”.
530	<code> res = agent.evaluate_suspect(</code>	Inside the current function/block, sets the variable <code>res</code> to the result of <code>agent.evaluate_suspect(</code> .
531	<code> name=name_str,</code>	Continuation line of the multi-line <code>res</code> assignment statement started above.
532	<code> behavior=behaviors,</code>	Continuation line of the multi-line <code>res</code> assignment statement started above.
533	<code> mo_suspected=behaviors,</code>	Continuation line of the multi-line <code>res</code> assignment statement started above.
534	<code> personality_notes=behaviors</code>	Continuation line of the multi-line <code>res</code> assignment statement started above.
535	<code>)</code>	Closes the multi-line <code>res</code> assignment opened above.
536	<code> if st.session_state.max_evals != "Unlimited":</code>	Conditional check — runs the indented block below only when <code>st.session_state.max_evals != "Unlimited"</code> evaluates to true.
537	<code> if</code>	Conditional check — runs the indented block below only when <code>st.session_state.quota_source == "trial"</code> evaluates to true.
538	<code> st.session_state.evals_left = _consume_trial(TRIAL_USER_ID)</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>evals_left</code> to <code>_consume_trial(TRIAL_USER_ID)</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
539	<code> else:</code>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
540	<code> st.session_state.evals_left = max(</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>evals_left</code> to <code>max(</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
541	<code> 0,</code>	Continuation line of the multi-line <code>session_state.evals_left</code> assignment statement started above.
542	<code> st.session_state.evals_left - 1</code>	Continuation line of the multi-line <code>session_state.evals_left</code> assignment statement started above.
543	<code>)</code>	Closes the multi-line <code>session_state.evals_left</code> assignment opened above.
544	<code> matched_cases = res.get("similar_cases", [])</code>	Inside the current function/block, sets the variable <code>matched_cases</code> to the result of <code>res.get("similar_cases", [])</code> .
545	<code> risk_lbl = res.get("risk_level", "UNKNOWN")</code>	Inside the current function/block, sets the variable <code>risk_lbl</code> to the result of <code>res.get("risk_level", "UNKNOWN")</code> .
546	<code> st.session_state.latest_results = {</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>latest_results</code> to <code>{</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
547	<code> "name": res.get("suspect_name", name_str),</code>	Supplies the <code>name</code> entry (<code>res.get("suspect_name", name_str),</code>) as part of the multi-line <code>session_state.latest_results</code> assignment started above.
548	<code> "age": age,</code>	Supplies the <code>age</code> entry (<code>age,</code>) as part of the multi-line <code>session_state.latest_results</code> assignment
549	<code> "behaviors": behaviors,</code>	Supplies the <code>behaviors</code> entry (<code>behaviors,</code>) as part of the multi-line <code>session_state.latest_results</code> assignment
550	<code> }</code>	Closes the multi-line <code>session_state.latest_results</code> assignment started above.

behaviors,	started above.
546 res.get("tendency_score", "0%"),	Supplies the tendency_score entry (`res.get("tendency_score", "0%"),`) as part of the multi-line session_state.latest_results assignment` started above.
547 "risk_level": risk_lbl,	Supplies the risk_level entry (risk_lbl,) as part of the multi-line session_state.latest_results assignment` started above.
548 "matched_cases": matched_cases,	Supplies the matched_cases entry (matched_cases,) as part of the multi-line session_state.latest_results assignment` started above.
549 res.get("summary", f"Suspect pattern evaluated for behavior traits: {behaviors[:60]}..."),	Supplies the summary_text entry (`res.get("summary", f"Suspect pattern evaluat...`) as part of the multi-line session_state.latest_results assignment` started above.
550 "timestamp": time.time()	Supplies the timestamp entry (time.time()) as part of the multi-line session_state.latest_results assignment` started above.
551 }	Closes the multi-line session_state.latest_results assignment` opened above.
552 st.session_state.analysis_history.append(st.session_state.latest_results.copy())	Renders a Streamlit append` element on the page.
553 except Exception as err:	Catches an error from the try` block above of type Exception as err, so the app can show a friendly message instead of crashing.
554 st.error(f"Analysis failed: {err}")	Renders a red error message box on the page, labelled/showing "Analysis failed: {err}"`.
555	Blank line — vertical spacing added purely for readability; has no functional effect.
556 if st.session_state.latest_results:	Conditional check — runs the indented block below only when st.session_state.latest_results` evaluates to true.
557 res = st.session_state.latest_results	Inside the current function/block, sets the variable res` to the result of st.session_state.latest_results`.
558 st.markdown(f"### Profile: **{res['name']}**")	Renders formatted Markdown/HTML text output on the page, labelled/showing "### Profile: **{res['name']}**"`.
559 m_coll, m_col2 = st.columns(2)	Splits the page horizontally into side-by-side columns, letting several pieces of content (e.g. a QR code and payment instructions) sit next to each other.
560 m_coll.metric("Tendency Score", str(res["tendency_score"]))	Calls m_coll.metric(...),` executing that function/method for its side effect (its return value, if any, is not stored).
561 m_col2.metric("Risk Level", res["risk_level"])	Calls m_col2.metric(...),` executing that function/method for its side effect (its return value, if any, is not stored).
562 st.info(res["summary_text"])	Renders a blue informational message box on the page, labelled/showing "summary_text"`.
563 st.markdown("#### Matched Precedents")	Renders formatted Markdown/HTML text output on the page, labelled/showing "#### Matched Precedents"`.
564 if res["matched_cases"]:	Conditional check — runs the indented block below only when `res["matched_cases"]` evaluates to true.
565 for case in res["matched_cases"]:	Loop — iterates case` over `res["matched_cases"]`, running the indented block once per item.
566 with st.expander(f"✳️ {case.get('case_title', 'Historical Precedent')} ({case.get('location', 'Global')})"):	Opens a collapsible expander panel; widgets inside the indented block are hidden until the user clicks to expand it.
567 st.write(f"***Case ID:** {case.get('case_id', 'N/A')}")	Renders generic auto-formatted output text on the page, labelled/showing "***Case ID:** {case.get('case_id', 'N/A')}"`.
568 st.write(f"***Details:** {case.get('summary', case.get('snippet', 'No snippet available.'))}")	Renders generic auto-formatted output text on the page, labelled/showing "***Details:** {case.get('summary', case.get('snippet', 'No sn...}"`.
569 else:	Fallback branch — runs when none of the preceding if / elif` conditions above were true.
570 st.info("No direct precedent matches found in ChromaDB above threshold.")	Renders a blue informational message box on the page, labelled/showing "No direct precedent matches found in ChromaDB above thresho..."`.
571 try:	Begins a try` block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
572 pdf_bytes = generate_pdf_report(	Inside the current function/block, sets the variable pdf_bytes` to the result of generate_pdf_report(`.
573 res["name"], res["age"], res["tendency_score"],	Continuation of the multi-line `pdf_bytes` assignment` above, contributing the text "name".
574 res["risk_level"], res["behaviors"], res["matched_cases"]	Continuation of the multi-line `pdf_bytes` assignment` above, contributing the text "risk_level".
575)	Closes the multi-line `pdf_bytes` assignment` opened above.
576 dynamic_key = f"d1_pdf_{int(res.get('timestamp', time.time()))}"	Inside the current function/block, sets the variable dynamic_key` to the result of `f"d1_pdf_{int(res.get('timestamp', time.time()))}"`.
577 st.download_button(	Renders a file-download button on the page.
578 label="📄 Download Executive PDF Report", data=pdf_bytes,	Continuation of the multi-line st.download_button(...)` call` above, contributing the text "📄 Download Executive PDF Report".
579 file_name=f"Profile_Report_{res['name'].replace(' ', '_')}.pdf",	Continuation of the multi-line st.download_button(...)` call` above, contributing the text "Profile_Report_{res['name'].replace(' ', '_')}.pdf".
580 mime="application/pdf", use_container_width=True, key=dynamic_key	Continuation of the multi-line st.download_button(...)` call` above, contributing the text "application/pdf".
581)	Closes the multi-line st.download_button(...)` call` opened above.
582 except Exception:	Catches an error from the try` block above of type Exception, so the app can show a friendly message instead of crashing.
583 st.caption("PDF generation ready.")	Renders small helper/caption text on the page, labelled/showing "PDF generation ready."`.
584	Blank line — vertical spacing added purely for readability; has no functional effect.
585 with tab_billing:	Enters the tab_billing` tab panel — everything indented below is rendered only while that tab is active.
586 PLANS = {	Starts a dictionary that defines the subscription plans (name → price / evaluation-count / display label) used to render the pricing cards shown to the user.
587 "🏆 Starter Agency": {"amount": 500, "evals": 100, "label": "₹500 / mo — 100 Evaluations"},	Continuation of the multi-line PLANS` dictionary (subscription pricing table)` above, contributing the text "🏆 Starter Agency".
588 "🏆 Pro Agency": {"amount": 1000, "evals": 500, "label": "₹1,000 / mo — 500 Evaluations"},	Continuation of the multi-line PLANS` dictionary (subscription pricing table)` above, contributing the text "🏆 Pro Agency".
589 "🏆 Enterprise SaaS": {"amount": 2000, "evals": "Unlimited", "label": "₹2,000 / mo — Unlimited Evaluations"},	Continuation of the multi-line PLANS` dictionary (subscription pricing table)` above, contributing the text "🏆 Enterprise SaaS".
590 }	Closes the multi-line PLANS` dictionary (subscription pricing table)` opened above.
591 st.subheader("Select Your Subscription Plan")	Renders a secondary section heading on the page, labelled/showing "Select Your Subscription Plan"`.

592	<code>st.caption("Quota is unlocked by the admin after UPI payment is verified.")</code>	Renders small helper/caption text on the page, labelled/showing “Quota is unlocked by the admin after UPI payment is verifie...”.
593	<code>p_col1, p_col2, p_col3 = st.columns(3)</code>	Splits the page horizontally into side-by-side columns, letting several pieces of content (e.g. a QR code and payment instructions) sit next to each other.
594	<code>for col, (plan_name, plan_info) in zip([p_col1, p_col2, p_col3], PLANS.items()):</code>	Loop — iterates <code>col, (plan_name, plan_info)</code> over <code>zip([p_col1, p_col2, p_col3], PLANS.items())</code> , running the indented block once per item.
595	<code>with col:</code>	Enters one of the layout columns created above (<code>col</code>) — every widget in the indented block is rendered inside that column.
596	<code>st.markdown(f"### {plan_name}")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “### {plan_name}”.
597	<code>st.markdown(f"***{plan_info['label']}***")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “***{plan_info['label']}***”.
598	<code>if plan_name == "🏡 Starter Agency":</code>	Conditional check — runs the indented block below only when <code>`plan_name == "🏡 Starter Agency"``</code> evaluates to true.
599	<code>st.markdown("** 100 Evaluations / mo\n* Standard RAG Precedent Search\n* Basic PDF Export")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “* 100 Evaluations / mo\n* Standard RAG Precedent Search\n* ...”.
600	<code>elif plan_name == "🏡 Pro Agency":</code>	Conditional check — runs the indented block below only when <code>`plan_name == "🏡 Pro Agency"``</code> evaluates to true.
601	<code>st.markdown("** 500 Evaluations / mo\n* Fast ChromaDB Vector Search\n* Custom JSON File Indexer")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “* 500 Evaluations / mo\n* Fast ChromaDB Vector Search\n* Cu...”.
602	<code>else:</code>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
603	<code>st.markdown("** Unlimited Evaluations\n* Private Vector Database\n* Dedicated API & Priority Support")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “* Unlimited Evaluations\n* Private Vector Database\n* Dedic...”.
604	<code>btn_key = f"plan_select_{plan_name.replace(' ','_').replace('/','_').replace('🏡','').replace('🏡','').replace('🏡','')}"</code>	Inside the current function/block, sets the variable <code>btn_key</code> to the result of <code>`f"plan_select_{plan_name.replace(' ','_').replace('/','_').replace('🏡','').replace('🏡','').replace('🏡','')}"`</code> .
605	<code>if st.button(f"Select {plan_name}", key=btn_key, use_container_width=True):</code>	Conditional check — runs the indented block below only when <code>`st.button(f"Select {plan_name}", key=btn_key, use_container_width=True)`</code> evaluates to true.
606	<code>st.session_state.pending_plan = plan_name</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_plan</code> to <code>plan_name</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
607	<code>st.session_state.pending_amount = plan_info["amount"]</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_amount</code> to <code>`plan_info["amount"]`</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
608	<code>st.session_state.pending_evals = plan_info["evals"]</code>	Updates Streamlit's persistent <code>session_state</code> — sets <code>pending_evals</code> to <code>`plan_info["evals"]`</code> , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
609	<code>st.rerun()</code>	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
610		Blank line — vertical spacing added purely for readability; has no functional effect.
611	<code>if st.session_state.pending_plan:</code>	Conditional check — runs the indented block below only when <code>st.session_state.pending_plan</code> evaluates to true.
612	<code>st.divider()</code>	Renders a horizontal divider line on the page.
613	<code>plan = st.session_state.pending_plan</code>	Inside the current function/block, sets the variable <code>plan</code> to the result of <code>st.session_state.pending_plan</code> .
614	<code>amount = st.session_state.pending_amount</code>	Inside the current function/block, sets the variable <code>amount</code> to the result of <code>st.session_state.pending_amount</code> .
615	<code>evals = st.session_state.pending_evals</code>	Inside the current function/block, sets the variable <code>evals</code> to the result of <code>st.session_state.pending_evals</code> .
616	<code>st.subheader(f"🏠 Complete Payment for {plan}")</code>	Renders a secondary section heading on the page, labelled/showing “🏠 Complete Payment for {plan}”.
617	<code>qr_col, inst_col = st.columns([1, 2])</code>	Splits the page horizontally into side-by-side columns, letting several pieces of content (e.g. a QR code and payment instructions) sit next to each other.
618	<code>with qr_col:</code>	Enters one of the layout columns created above (<code>qr_col</code>) — every widget in the indented block is rendered inside that column.
619	<code>try:</code>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
620	<code>qr_bytes = make_upi_qr(amount, f"Detective_AI_{plan.split()[-1]}")</code>	Inside the current function/block, sets the variable <code>qr_bytes</code> to the result of <code>`make_upi_qr(amount, f"Detective_AI_{plan.split()[-1]}")`</code> .
621	<code>st.image(qr_bytes, caption=f"Scan to Pay ₹ {amount:,}", width=220)</code>	Renders an image element on the page, labelled/showing “Scan to Pay ₹{amount:,}”.
622	<code>except Exception as e:</code>	Catches an error from the <code>try</code> block above of type <code>Exception</code> as <code>e</code> , so the app can show a friendly message instead of crashing.
623	<code>st.warning(f"QR could not be generated: {e}")</code>	Renders an amber warning message box on the page, labelled/showing “QR could not be generated: {e}”.
624	<code>st.code(</code>	Renders a monospaced code block on the page.
625	<code>f"upi://pay?pa={UPI_VPA}&pn=Aditya%20Srivastava"</code>	Continuation of the multi-line <code>st.code(...)</code> call above, contributing the text <code>"upi://pay?pa={UPI_VPA}&pn=Aditya%20Srivastava"</code> .
626	<code>f"&m={amount}&cu=INR&tn=Detective_AI_{plan.split()[-1]}",</code>	Continuation of the multi-line <code>st.code(...)</code> call above, contributing the text <code>"&m={amount}&cu=INR&tn=Detective_AI_{plan.split()[-1]}"</code> .
627	<code>language="text"</code>	Continuation of the multi-line <code>st.code(...)</code> call above, contributing the text <code>"text"</code> .
628	<code>)</code>	Closes the multi-line <code>st.code(...)</code> call opened above.
629	<code>with inst_col:</code>	Enters one of the layout columns created above (<code>inst_col</code>) — every widget in the indented block is rendered inside that column.
630	<code>st.markdown(f"""</code>	Renders formatted Markdown/HTML text output on the page.
631	<code>**Amount Payable:** ₹{amount:,}</code>	Continuation line of the multi-line <code>st.markdown(...)</code> call statement started above.
632	<code>**UPI ID:** `{UPI_VPA}`</code>	Continuation line of the multi-line <code>st.markdown(...)</code> call statement started above.
633	<code>**Payee Name:** Aditya Srivastava</code>	Continuation line of the multi-line <code>st.markdown(...)</code> call statement started above.
634		Blank line inside a multi-line statement — has no runtime effect.
635	<code>**Steps:**</code>	Continuation line of the multi-line <code>st.markdown(...)</code> call statement started above.
636	<code>1. Open PhonePe / Google Pay / Paytm</code>	Continuation line of the multi-line <code>st.markdown(...)</code> call statement started above.
637	<code>2. Scan the QR code or pay to UPI ID above</code>	Continuation line of the multi-line <code>st.markdown(...)</code> call statement started above.
638	<code>3. Note the **12-digit UTR / Transaction Reference** shown in your payment app</code>	Continuation line of the multi-line <code>st.markdown(...)</code> call statement started above.
639	<code>4. Enter it below and click **Submit Proof**</code>	Continuation line of the multi-line <code>st.markdown(...)</code> call statement started above.
640	<code>""")</code>	Continuation line of the multi-line <code>st.markdown(...)</code> call statement started above.
641	<code>with st.form(key=f"utr_form_{plan.replace('</code>	Opens a Streamlit form container — inputs placed inside it are batched together and only submitted when its submit button is

'','_').replace('/','_')}"):	pressed.
642utr_input = st.text_input("Enter UTR / Transaction Reference ID", placeholder="e.g. 426789012345", max_chars=30)	Renders a single-line text input box on the page, labelled/showing "Enter UTR / Transaction Reference ID". The value the user enters/selects is stored in utr_input for use later in the script.
643paid_input = st.text_input("Amount You Paid (₹)", placeholder=f"e.g. {amount}", max_chars=10)	Renders a single-line text input box on the page, labelled/showing "Amount You Paid (₹)". The value the user enters/selects is stored in paid_input for use later in the script.
644submit utr = st.form_submit_button("📄 Submit Payment Proof", use_container_width=True)	Renders a submit button bound to the enclosing form on the page, labelled/showing "📄 Submit Payment Proof". The value the user enters/selects is stored in submit utr for use later in the script.
645if submit utr:	Conditional check — runs the indented block below only when submit utr evaluates to true.
646if not utr_input.strip():	Conditional check — runs the indented block below only when not utr_input.strip() evaluates to true.
647st.error("Please enter your UTR / Transaction Reference.")	Renders a red error message box on the page, labelled/showing "Please enter your UTR / Transaction Reference.".
648elif not paid_input.strip():	Conditional check — runs the indented block below only when not paid_input.strip() evaluates to true.
649st.error("Please enter the amount you paid.")	Renders a red error message box on the page, labelled/showing "Please enter the amount you paid.".
650else:	Fallback branch — runs when none of the preceding if / elif conditions above were true.
651try:	Begins a try block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
652paid_val = float(paid_input.replace(",","").strip())	Inside the current function/block, sets the variable paid_val to the result of `float(paid_input.replace(",","").strip())`.
653except ValueError:	Catches an error from the try block above of type ValueError, so the app can show a friendly message instead of crashing.
654st.error("Invalid amount — enter a number.")	Renders a red error message box on the page, labelled/showing "Invalid amount — enter a number.".
655paid_val = None	Inside the current function/block, sets the variable paid_val to the result of None.
656if paid_val is not None:	Conditional check — runs the indented block below only when paid_val is not None evaluates to true.
657_s, _m, _r = submit_payment(	Inside the current function/block, sets the variable _s, _m, _r to the result of submit_payment(.
658plan=plan, required_amount=float(amount),	Continuation line of the multi-line ` _s, _m, _r assignment ` statement started above.
659amount_paid=paid_val, evals=evals, utr=utr_input.strip()	Continuation line of the multi-line ` _s, _m, _r assignment ` statement started above.
660)	Closes the multi-line ` _s, _m, _r assignment ` opened above.
661if _s == STATUS_FLAGGED:	Conditional check — runs the indented block below only when _s == STATUS_FLAGGED evaluates to true.
662st.error(_m)	Renders a red error message box on the page.
663elif _s == STATUS_PARTIAL:	Conditional check — runs the indented block below only when _s == STATUS_PARTIAL evaluates to true.
664st.warning(_m)	Renders an amber warning message box on the page.
665utr_input.strip()	Updates Streamlit's persistent session_state — sets partial utr to utr_input.strip() , so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
666None	Updates Streamlit's persistent session_state — sets pending_plan to None, so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
667= None	Updates Streamlit's persistent session_state — sets pending_amount to None, so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
668= None	Updates Streamlit's persistent session_state — sets pending_evals to None, so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
669st.rerun()	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
670else:	Fallback branch — runs when none of the preceding if / elif conditions above were true.
671st.success("✓ Payment proof submitted! Your quota will be unlocked after admin verification.")	Renders a green success message box on the page, labelled/showing "✓ Payment proof submitted! Your quota will be unlocked after...".
672None	Updates Streamlit's persistent session_state — sets pending_plan to None, so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
673= None	Updates Streamlit's persistent session_state — sets pending_amount to None, so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
674= None	Updates Streamlit's persistent session_state — sets pending_evals to None, so this value survives page reruns (Streamlit's mechanism for keeping frontend state between interactions).
675st.rerun()	Forces Streamlit to immediately re-execute the whole script from the top, redrawing the UI with the latest session state — this is how the app reacts instantly to a button click.
676	Blank line — vertical spacing added purely for readability; has no functional effect.
677with tab_contact:	Enters the tab_contact tab panel — everything indented below is rendered only while that tab is active.
678st.subheader("📞 Contact, Feedback & Feature Requests")	Renders a secondary section heading on the page, labelled/showing "📞 Contact, Feedback & Feature Requests".
679st.markdown("Have a question, found a bug, or want to suggest a new feature? Reach out directly — every message is read personally.")	Renders formatted Markdown/HTML text output on the page.
680st.divider()	Renders a horizontal divider line on the page.
681c1, c2 = st.columns(2)	Splits the page horizontally into side-by-side columns, letting several pieces of content (e.g. a QR code and payment instructions) sit next to each other.
682with c1:	Enters one of the layout columns created above (c1) — every widget in the indented block is rendered inside that column.
683st.markdown("### 🏠 Founders — Direct Contact")	Renders formatted Markdown/HTML text output on the page, labelled/showing "### 🏠 Founders — Direct Contact".
684st.markdown("***Aditya Srivastava** \n[Founder] \n📧 [yeahboyadi@gmail.com] (mailto:yeahboyadi@gmail.com)")	Renders formatted Markdown/HTML text output on the page, labelled/showing "***Aditya Srivastava** \n[Founder] \n📧 [yeahboyadi@gmail.co...".
685st.markdown("***Akshat Verma** \n[Partner] \n📧 [akshat.v2166@gmail.com] (mailto:akshat.v2166@gmail.com)")	Renders formatted Markdown/HTML text output on the page, labelled/showing "***Akshat Verma** \n[Partner] \n📧 [akshat.v2166@gmail.com)](...".
686st.markdown("...")	Renders formatted Markdown/HTML text output on the page, labelled/showing "...".
687st.markdown("### 🐛 Bug Reports")	Renders formatted Markdown/HTML text output on the page, labelled/showing "### 🐛 Bug Reports".
688st.markdown("Please include: \n- What you were	Renders formatted Markdown/HTML text output on the page

687	<pre>def main(): """Main function to run the app. Make your code doing \n- What error / unexpected behaviour appeared \n- Screenshot if possible \n\nSend to ** [yeahboyadi@gmail.com] (mailto:yeahboyadi@gmail.com)** or ** [akshat.v2166@gmail.com] (mailto:akshat.v2166@gmail.com)** with subject line: `[BUG] Detective AI - <short     description>`")</pre>	Renders formatted Markdown/HTML text output on the page.
689	<pre>with c2:</pre>	Enters one of the layout columns created above (<code>c2</code>) — every widget in the indented block is rendered inside that column.
690	<pre>st.markdown("### 📝 Suggest a Feature")</pre>	Renders formatted Markdown/HTML text output on the page, labelled/showing “ <code>### 📝 Suggest a Feature</code> ”.
691	<pre>st.markdown("Ideas for new capabilities are welcome: \n- New case-matching algorithms \n- Additional report formats \n- Integrations (WhatsApp alerts, CRM sync, etc.) \n\nSend to **[yeahboyadi@gmail.com] (mailto:yeahboyadi@gmail.com)** or ** [akshat.v2166@gmail.com] (mailto:akshat.v2166@gmail.com)** with subject line: `[FEATURE REQUEST] <your idea>`")</pre>	Renders formatted Markdown/HTML text output on the page.
692	<pre>st.markdown("...")</pre>	Renders formatted Markdown/HTML text output on the page, labelled/showing “ <code>...</code> ”.
693	<pre>st.markdown("### 🛡️ Privacy & Data")</pre>	Renders formatted Markdown/HTML text output on the page, labelled/showing “ <code>### 🛡️ Privacy & Data</code> ”.
694	<pre>st.markdown("All suspect profiling data is processed locally in your session. \nNo case data is stored on our servers without your explicit upload. \nPayments are verified manually — we never store card details.")</pre>	Renders formatted Markdown/HTML text output on the page.
695	<pre>st.divider()</pre>	Renders a horizontal divider line on the page.
696	<pre>st.info()</pre>	Renders a blue informational message box on the page.
697	<pre>"⌚ **Response time:** Typically within 24 hours on weekdays. \n"</pre>	Continuation of the multi-line <code>st.info(...)</code> call above, contributing the text “ <code>⌚ **Response time:** Typically within 24 hours on wee...</code> ”.
698	<pre>#f🌐 **Platform:** {PLATFORM_URL}"</pre>	Continuation of the multi-line <code>st.info(...)</code> call above, contributing the text “ <code>🌐 **Platform:** {PLATFORM_URL}</code> ”.
699	<pre>)</pre>	Closes the multi-line <code>st.info(...)</code> call opened above.
700		Blank line — vertical spacing added purely for readability; has no functional effect.
701	<pre>if st.session_state.is_admin:</pre>	Conditional check — runs the indented block below only when <code>st.session_state.is_admin</code> evaluates to true.
702	<pre>with tab_outreach:</pre>	Enters the <code>tab_outreach</code> tab panel — everything indented below is rendered only while that tab is active.
703	<pre>st.subheader("🔍 B2B Lead Scraper & Cold Email Outreach")</pre>	Renders a secondary section heading on the page, labelled/showing “ <code>🔍 B2B Lead Scraper & Cold Email Outreach</code> ”.
704	<pre>st.caption("Scrape target agency/detective contact information, select specific prospects, preview emails, and dispatch only after approval.")</pre>	Renders small helper/caption text on the page.
705		Blank line — vertical spacing added purely for readability; has no functional effect.
706	<pre>col_kw, col_loc = st.columns(2)</pre>	Splits the page horizontally into side-by-side columns, letting several pieces of content (e.g. a QR code and payment instructions) sit next to each other.
707	<pre>with col_kw:</pre>	Enters a <code>with</code> context (<code>col_kw</code>) that scopes the indented block below.
708	<pre>target_keyword = st.text_input("Target Keyword / Niche", value="Detective Agency", key="outreach_kw")</pre>	Renders a single-line text input box on the page, labelled/showing “Target Keyword / Niche”. The value the user enters/selects is stored in <code>target_keyword</code> for use later in the script.
709	<pre>with col_loc:</pre>	Enters a <code>with</code> context (<code>col_loc</code>) that scopes the indented block below.
710	<pre>target_location = st.text_input("Location", value="Delhi", key="outreach_loc")</pre>	Renders a single-line text input box on the page, labelled/showing “Location”. The value the user enters/selects is stored in <code>target_location</code> for use later in the script.
711		Blank line — vertical spacing added purely for readability; has no functional effect.
712	<pre># Option to allow generated placeholder leads (disabled by default)</pre>	Comment (not executed): “Option to allow generated placeholder leads (disabled by default)” — documents intent for the code that follows.
713	<pre>allow_generated = st.checkbox("Allow generated placeholder leads when public data is sparse (verify manually)", value=False, key="chk_allow_generated")</pre>	Renders a checkbox toggle on the page, labelled/showing “Allow generated placeholder leads when public data is spars...”. The value the user enters/selects is stored in <code>allow_generated</code> for use later in the script.
714	<pre># Minimum confidence filter for displayed leads (0- 100). Default 60 hides low-confidence/generated leads.</pre>	Comment (not executed): “Minimum confidence filter for displayed leads (0-100). Default 60 hides low-confidence/generated leads.” — documents intent for the code that follows.
715	<pre>min_conf = st.slider("Minimum confidence to display leads", 0, 100, 60, key="lead_min_confidence")</pre>	Renders a draggable numeric slider on the page, labelled/showing “Minimum confidence to display leads”. The value the user enters/selects is stored in <code>min_conf</code> for use later in the script.
716	<pre>if st.button("🔎 Scrape Leads", type="primary", use_container_width=True, key="btn_scrape_leads"):</pre>	Conditional check — runs the indented block below only when “ <code>st.button("🔎 Scrape Leads", type="primary", use_container_width=True, key="btn_scrape_leads"):</code> ” evaluates to true.
717	<pre>with st.spinner("Scraping leads across web sources..."):</pre>	Shows a loading spinner (“Scraping leads across web sources...”) while the indented block runs, giving the user visual feedback that work is in progress.
718	<pre>try:</pre>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
719	<pre>scraped_data = scrape_leads_sync(target_keyword, target_location, max_results=20, allow_generated=allow_generated)</pre>	Inside the current function/block, sets the variable <code>scraped_data</code> to the result of <code>scrape_leads_sync(target_keyword, target_location, max_results=20, allow_generated=allow_generated)</code> .
720	<pre>if scraped_data:</pre>	Conditional check — runs the indented block below only when <code>scraped_data</code> evaluates to true.
721	<pre># Show scraped results (restore original behaviour: display all scraped leads)</pre>	Comment (not executed): “Show scraped results (restore original behaviour: display all scraped leads)” — documents intent for the code that follows.
722	<pre>displayed = scraped_data</pre>	Inside the current function/block, sets the variable <code>displayed</code> to the result of <code>scraped_data</code> .
723	<pre>try:</pre>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
724	<pre># still compute confidence column for info if present</pre>	Comment (not executed): “still compute confidence column for info if present” — documents intent for the code that follows.
725	<pre>scraped_df = pd.DataFrame(scraped_data)</pre>	Inside the current function/block, sets the variable <code>scraped_df</code> to the result of <code>pd.DataFrame(scraped_data)</code> .
726	<pre>scraped_df['confidence'] = scraped_df.get('confidence', 50).fillna(50)</pre>	Inside the current function/block, sets the variable <code>scraped_df['confidence']</code> to the result of <code>scraped_df.get('confidence', 50).fillna(50)</code> .
727	<pre>except Exception:</pre>	Catches an error from the <code>try</code> block above of type <code>Exception</code> , so the app can show a friendly message instead of crashing.
728	<pre>pass</pre>	No-op placeholder — deliberately does nothing (satisfies Python's block-body requirement).
729		Blank line — vertical spacing added purely for readability; has no functional effect.
730	<pre>st.success(f"Successfully scraped {len(scraped_data)} leads!")</pre>	Renders a green success message box on the page, labelled/showing “Successfully scraped {len(scraped_data)} leads!”.

<pre>{len(scraped_data)} leads!")</pre>	
<pre>731 st.dataframe(pd.DataFrame(displayed), use_container_width=True)</pre>	Renders a scrollable, sortable data table on the page.
<pre>732 else:</pre>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
<pre>733 st.info("No leads found matching your criteria.")</pre>	Renders a blue informational message box on the page, labelled/showing "No leads found matching your criteria."
<pre>734 except Exception as e:</pre>	Catches an error from the <code>try</code> block above of type <code>Exception</code> as <code>e</code> , so the app can show a friendly message instead of crashing.
<pre>735 st.error(f"Scraping failed: {e}")</pre>	Renders a red error message box on the page, labelled/showing "Scraping failed: {e}".
<pre>736</pre>	Blank line — vertical spacing added purely for readability; has no functional effect.
<pre>737 st.divider()</pre>	Renders a horizontal divider line on the page.
<pre>738 st.markdown("### 📧 Email Dispatcher")</pre>	Renders formatted Markdown/HTML text output on the page, labelled/showing "### 📧 Email Dispatcher".
<pre>739 saved_leads = load_leads()</pre>	Inside the current function/block, sets the variable <code>saved_leads</code> to the result of <code>load_leads()</code> .
<pre>740</pre>	Blank line — vertical spacing added purely for readability; has no functional effect.
<pre>741</pre>	Blank line — vertical spacing added purely for readability; has no functional effect.
<pre>742 if saved_leads:</pre>	Conditional check — runs the indented block below only when <code>saved_leads</code> evaluates to true.
<pre>743 # Apply confidence filter to saved_leads for display/selection (non-destructive)</pre>	Comment (not executed): "Apply confidence filter to saved_leads for display/selection (non-destructive)" — documents intent for the code that follows.
<pre>744 min_conf_local = st.session_state.get("lead_min_confidence", 60)</pre>	Renders a Streamlit <code>get</code> element on the page, labelled/showing "lead_min_confidence". The value the user enters/selects is stored in <code>min_conf_local</code> for use later in the script.
<pre>745 saved_leads_display = []</pre>	Inside the current function/block, sets the variable <code>saved_leads_display</code> to the result of <code>[]</code> .
<pre>746 for lead in saved_leads:</pre>	Loop — iterates <code>lead</code> over <code>saved_leads</code> , running the indented block once per item.
<pre>747 if not isinstance(lead, dict):</pre>	Conditional check — runs the indented block below only when <code>not isinstance(lead, dict)</code> evaluates to true.
<pre>748 continue</pre>	Executes: <code>continue</code> — supporting statement within the current block.
<pre>749 try:</pre>	Begins a <code>try</code> block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
<pre>750 conf = int(lead.get('confidence', 50))</pre>	Inside the current function/block, sets the variable <code>conf</code> to the result of <code>int(lead.get('confidence', 50))</code> .
<pre>751 except Exception:</pre>	Catches an error from the <code>try</code> block above of type <code>Exception</code> , so the app can show a friendly message instead of crashing.
<pre>752 conf = 50</pre>	Inside the current function/block, sets the variable <code>conf</code> to the result of <code>50</code> .
<pre>753 verified = str(lead.get('verification_status', '')).strip().lower() == 'verified'</pre>	Inside the current function/block, sets the variable <code>verified</code> to the result of <code>str(lead.get('verification_status', '')).strip().lower() == 'verified'</code> .
<pre>754 if conf >= int(min_conf_local) or verified:</pre>	Conditional check — runs the indented block below only when <code>conf >= int(min_conf_local)</code> or <code>verified</code> evaluates to true.
<pre>755 saved_leads_display.append(lead)</pre>	Calls <code>saved_leads_display.append(lead)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
<pre>756</pre>	Blank line — vertical spacing added purely for readability; has no functional effect.
<pre>757 filtered_out = len(saved_leads) - len(saved_leads_display)</pre>	Inside the current function/block, sets the variable <code>filtered_out</code> to the result of <code>len(saved_leads) - len(saved_leads_display)</code> .
<pre>758 if filtered_out > 0:</pre>	Conditional check — runs the indented block below only when <code>filtered_out > 0</code> evaluates to true.
<pre>759 st.info(f"{filtered_out} leads hidden by confidence filter (min {min_conf_local}). Uncheck the filter to view all.")</pre>	Renders a blue informational message box on the page.
<pre>760</pre>	Blank line — vertical spacing added purely for readability; has no functional effect.
<pre>761 email_ready_count = sum(</pre>	Inside the current function/block, sets the variable <code>email_ready_count</code> to the result of <code>sum(</code> .
<pre>762 1 for lead in saved_leads_display</pre>	Continuation line of the multi-line <code>email_ready_count</code> assignment` statement started above.
<pre>763 if isinstance(lead, dict) and re.fullmatch(</pre>	Continuation line of the multi-line <code>email_ready_count</code> assignment` statement started above.
<pre>764 r"[^\s@]+@[^\s@]+\.[^\s@]+",</pre>	Continuation of the multi-line <code>email_ready_count</code> assignment` above, contributing the text <code>"[^\s@]+@[^\s@]+\.[^\s@]+".</code>
<pre>765 str(</pre>	Continuation line of the multi-line <code>email_ready_count</code> assignment` statement started above.
<pre>766 lead.get("contact_email")</pre>	Continuation of the multi-line <code>email_ready_count</code> assignment` above, contributing the text "contact_email".
<pre>767 or lead.get("email")</pre>	Continuation of the multi-line <code>email_ready_count</code> assignment` above, contributing the text "email".
<pre>768 or lead.get("email_address")</pre>	Continuation of the multi-line <code>email_ready_count</code> assignment` above, contributing the text "email_address".
<pre>769 or ""</pre>	Continuation line of the multi-line <code>email_ready_count</code> assignment` statement started above.
<pre>770).strip(),</pre>	Continuation line of the multi-line <code>email_ready_count</code> assignment` statement started above.
<pre>771)</pre>	Closes the multi-line <code>email_ready_count</code> assignment` opened above.
<pre>772)</pre>	Closes the multi-line <code>email_ready_count</code> assignment` opened above.
<pre>773 st.write(</pre>	Renders generic auto-formatted output text on the page.
<pre>774 f"Loaded **{len(saved_leads)}** prospects — **{len(saved_leads_display)}** shown (min confidence {min_conf_local}) — "</pre>	Continuation line of the multi-line <code>st.write(...)</code> call statement started above.
<pre>775 f"**{email_ready_count}** have a valid email address."</pre>	Continuation of the multi-line <code>st.write(...)</code> call above, contributing the text <code>"**{email_ready_count}** have a valid email address."</code> .
<pre>776)</pre>	Closes the multi-line <code>st.write(...)</code> call opened above.
<pre>777 if email_ready_count == 0:</pre>	Conditional check — runs the indented block below only when <code>email_ready_count == 0</code> evaluates to true.
<pre>778 st.warning(</pre>	Renders an amber warning message box on the page.
<pre>779 "No verified email addresses are available yet. "</pre>	Continuation of the multi-line <code>st.warning(...)</code> call above, contributing the text "No verified email addresses are available yet."
<pre>780 "Prospects without a published email are kept for manual "</pre>	Continuation of the multi-line <code>st.warning(...)</code> call above, contributing the text "Prospects without a published email are kept for manual".
<pre>781 "enrichment and will not be emailed."</pre>	Continuation of the multi-line <code>st.warning(...)</code> call above, contributing the text "enrichment and will not be emailed."
<pre>782)</pre>	Closes the multi-line <code>st.warning(...)</code> call opened above.

783		Blank line — vertical spacing added purely for readability; has no functional effect.
784	<code>def _lead_label(lead, index):</code>	Defines the function <code>_lead_label(...)</code> , which builds a human-readable display label (name — email) for one scraped lead.
785	<code> if not isinstance(lead, dict):</code>	Conditional check — runs the indented block below only when <code>not isinstance(lead, dict)</code> evaluates to true.
786	<code> return f"Prospect {index + 1}"</code>	Returns <code>`f"Prospect {index + 1}"`</code> as the result of the current function, ending its execution.
787	<code> name = (</code>	Inside the current function/block, sets the variable <code>name</code> to the result of <code>(.</code>
788	<code> lead.get("agency_name") or</code> <code> lead.get("company_name") or</code>	Continuation of the multi-line <code>`name`</code> assignment` above, contributing the text “agency_name”.
789	<code> lead.get("company") or</code> <code> lead.get("business_name") or</code>	Continuation of the multi-line <code>`name`</code> assignment` above, contributing the text “company”.
790	<code> lead.get("name") or lead.get("title")</code> <code> or f"Prospect {index + 1}"</code>	Continuation of the multi-line <code>`name`</code> assignment` above, contributing the text “name”.
791	<code>)</code>	Closes the multi-line <code>`name`</code> assignment` opened above.
792	<code> email = lead.get("email") or</code> <code> lead.get("email_address") or lead.get("contact_email") or</code> <code> ""</code>	Inside the current function/block, sets the variable <code>email</code> to the result of <code>`lead.get("email") or lead.get("email_address") or lead.get("contact_e..."`</code> .
793	<code> name, email = str(name).strip(),</code> <code> str(email).strip()</code>	Inside the current function/block, sets the variable <code>name, email</code> to the result of <code>str(name).strip(), str(email).strip() .</code>
794	<code> return f"{name} — {email}" if email else</code> <code> name</code>	Returns <code>`f"{name} — {email}" if email else name`</code> as the result of the current function, ending its execution.
795		Blank line — vertical spacing added purely for readability; has no functional effect.
796	<code> lead_labels = [_lead_label(lead, index) for</code> <code> index, lead in enumerate(saved_leads_display)]</code>	Inside the current function/block, sets the variable <code>lead_labels</code> to the result of <code>[_lead_label(lead, index) for index, lead in enumerate(saved_leads_display) .</code>
797		Blank line — vertical spacing added purely for readability; has no functional effect.
798	<code> st.markdown("#### ☹ Choose Recipients")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “#### ☹ Choose Recipients”.
799	<code> st.caption("Use a command such as `send to</code> <code>    Delhi Inquiry Bureau`, `send to ABC and XYZ`, or `send to</code> <code>    all`.")</code>	Renders small helper/caption text on the page.
800		Blank line — vertical spacing added purely for readability; has no functional effect.
801	<code> selection_command = st.text_input(</code>	Renders a single-line text input box on the page. The value the user enters/selects is stored in <code>selection_command</code> for use later in the script.
802	<code> "Recipient Selection Command",</code>	Continuation of the multi-line <code>st.text_input(...)</code> call above, contributing the text “Recipient Selection Command”.
803	<code> placeholder="e.g. send to Delhi Inquiry</code> <code> Bureau and ABC Agency",</code>	Continuation of the multi-line <code>st.text_input(...)</code> call above, contributing the text “e.g. send to Delhi Inquiry Bureau and ABC Agency”.
804	<code> key="outreach_selection_command"</code>	Continuation of the multi-line <code>st.text_input(...)</code> call above, contributing the text “outreach_selection_command”.
805	<code>)</code>	Closes the multi-line <code>st.text_input(...)</code> call opened above.
806		Blank line — vertical spacing added purely for readability; has no functional effect.
807	<code> command_matches, command_not_found = [], []</code>	Inside the current function/block, sets the variable <code>command_matches, command_not_found</code> to the result of <code>[], [] .</code>
808	<code> if selection_command.strip():</code>	Conditional check — runs the indented block below only when <code>selection_command.strip()</code> evaluates to true.
809	<code> command_lower =</code> <code> selection_command.lower().strip()</code>	Inside the current function/block, sets the variable <code>command_lower</code> to the result of <code>selection_command.lower().strip() .</code>
810	<code> if command_lower in {"all", "send all",</code> <code>"send to all", "email all", "email everyone", "send to</code> <code>everyone"}:</code>	Conditional check — runs the indented block below only when <code>`command_lower in {"all", "send all", "send to all", "email all", "email everyone..."`</code> evaluates to true.
811	<code> command_matches = list(lead_labels)</code>	Inside the current function/block, sets the variable <code>command_matches</code> to the result of <code>list(lead_labels) .</code>
812	<code> else:</code>	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
813	<code> cleaned = re.sub(r"^(please\s+)?</code> <code>(send email mail)(\s+to)?\s*", "", command_lower,</code> <code> flags=re.I)</code>	Inside the current function/block, sets the variable <code>cleaned</code> to the result of <code>`re.sub(r"^(please\s+)?(send email mail)(\s+to)?\s*", "", command_lower..."`</code> .
814	<code> cleaned =</code> <code> re.sub(r"^(agencies leads recipients)\s*[:=-]?~\s*", "",</code> <code> cleaned, flags=re.I)</code>	Inside the current function/block, sets the variable <code>cleaned</code> to the result of <code>`re.sub(r"^(agencies leads recipients)\s*[:=-]?~\s*", "", cleaned, flag..."`</code> .
815	<code> requested_names = [p.strip() for p in</code> <code> re.split(r"\s*,\s* \s+and\s+", cleaned) if p.strip()]</code>	Inside the current function/block, sets the variable <code>requested_names</code> to the result of <code>[p.strip() for p in re.split(r"\s*,\s* \s+and\s+", cleaned) if p.strip..."`</code> .
816	<code> normalized_labels = [re.sub(r"^[a-z0-</code> <code>9@.]+", " ", label.lower()).strip() for label in</code> <code> lead_labels]</code>	Inside the current function/block, sets the variable <code>normalized_labels</code> to the result of <code>`[re.sub(r"^[a-z0-9@.]+", " ", label.lower()).strip() for label in lea..."`</code> .
817	<code> for requested in requested_names:</code>	Loop — iterates <code>requested</code> over <code>requested_names</code> , running the indented block once per item.
818	<code> req = re.sub(r"[a-z0-9@.]+", " ",</code> <code> requested.lower()).strip()</code>	Inside the current function/block, sets the variable <code>req</code> to the result of <code>`re.sub(r"[a-z0-9@.]+", " ", requested.lower()).strip()"`</code> .
819	<code> found = False</code>	Inside the current function/block, sets the variable <code>found</code> to the result of <code>False</code> .
820	<code> for label, norm in zip(lead_labels,</code> <code> normalized_labels):</code>	Loop — iterates <code>label, norm</code> over <code>zip(lead_labels, normalized_labels)</code> , running the indented block once per item.
821	<code> if req in norm or norm in req:</code>	Conditional check — runs the indented block below only when <code>req in norm or norm in req</code> evaluates to true.
822	<code> if label not in</code> <code> command_matches:</code>	Conditional check — runs the indented block below only when <code>label not in command_matches</code> evaluates to true.
823	<code> command_matches.append(label)</code>	Calls <code>command_matches.append(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
824	<code> found = True</code>	Inside the current function/block, sets the variable <code>found</code> to the result of <code>True</code> .
825	<code> if not found:</code>	Conditional check — runs the indented block below only when <code>not found</code> evaluates to true.
826	<code> command_not_found.append(requested)</code>	Calls <code>command_not_found.append(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
827		Blank line — vertical spacing added purely for readability; has no functional effect.
828	<code> if command_matches:</code>	Conditional check — runs the indented block below only when <code>command_matches</code> evaluates to true.
829	<code> st.success(f"Command matched **</code>	Renders a green success message box on the page, labelled/showing “Command matched **{len(command_matches)}**

<code>{len(command_matches)** recipient(s).")</code>	recipient(s).".
830 <code>if command_not_found:</code>	Conditional check — runs the indented block below only when <code>command_not_found</code> evaluates to true.
831 <code>st.warning("No matching recipient found for: " + ", ".join(command_not_found))</code>	Renders an amber warning message box on the page, labelled/showing “No matching recipient found for:”.
832	Blank line — vertical spacing added purely for readability; has no functional effect.
833 <code>if selection_command.strip():</code>	Conditional check — runs the indented block below only when <code>selection_command.strip()</code> evaluates to true.
834 <code>last_command = st.session_state.get("_outreach_last_selection_command")</code>	Renders a Streamlit <code>get</code> element on the page, labelled/showing “_outreach_last_selection_command”. The value the user enters/selects is stored in <code>last_command</code> for use later in the script.
835 <code>if last_command != selection_command:</code>	Conditional check — runs the indented block below only when <code>last_command != selection_command</code> evaluates to true.
836 <code>st.session_state["outreach_selected_labels"] = list(command_matches)</code>	Updates <code>`session_state["outreach_selected_labels"]`</code> to <code>list(command_matches)</code> , persisting this UI state across reruns.
837 <code>st.session_state["_outreach_last_selection_command"] = selection_command</code>	Updates <code>`session_state["_outreach_last_selection_command"]`</code> to <code>selection_command</code> , persisting this UI state across reruns.
838	Blank line — vertical spacing added purely for readability; has no functional effect.
839 <code>selected_labels = st.multiselect(840 "Select recipients to email", 841 options=lead_labels, 842 key="outreach_selected_labels" 843)</code>	Renders a multi-choice dropdown on the page. The value the user enters/selects is stored in <code>selected_labels</code> for use later in the script.
	Continuation of the multi-line <code>st.multiselect(...)</code> call above, contributing the text “Select recipients to email”.
	Continuation line of the multi-line <code>st.multiselect(...)</code> call statement started above.
	Continuation of the multi-line <code>st.multiselect(...)</code> call above, contributing the text “outreach_selected_labels”.
	Closes the multi-line <code>st.multiselect(...)</code> call opened above.
844 <code>selected_indices = [i for i, label in enumerate(lead_labels) if label in selected_labels]</code>	Inside the current function/block, sets the variable <code>selected_indices</code> to the result of <code>[i for i, label in enumerate(lead_labels) if label in selected_labels]</code> .
845 <code>selected_leads = [saved_leads_display[i] for i in selected_indices]</code>	Inside the current function/block, sets the variable <code>selected_leads</code> to the result of <code>[saved_leads_display[i] for i in selected_indices]</code> .
846 <code>st.info(f"Selected **{len(selected_leads)}** of **{len(saved_leads_display)}** prospects.")</code>	Renders a blue informational message box on the page, labelled/showing “Selected **{len(selected_leads)}** of **{len(saved_leads_display)}** prospects.”.
847	Blank line — vertical spacing added purely for readability; has no functional effect.
848 <code>st.markdown("#### 📧 Gmail Authentication")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “#### 📧 Gmail Authentication”.
849 <code>st.caption(850 "Enter the Gmail App Password for the sending account. " 851 "It is used only for this session and is not saved to your project files." 852)</code>	Renders small helper/caption text on the page.
	Continuation of the multi-line <code>st.caption(...)</code> call above, contributing the text “Enter the Gmail App Password for the sending account.”.
	Continuation of the multi-line <code>st.caption(...)</code> call above, contributing the text “It is used only for this session and is not saved to y...”.
	Closes the multi-line <code>st.caption(...)</code> call opened above.
853 <code>gmail_session_password = st.text_input(854 "Gmail App Password", 855 type="password", 856 placeholder="Enter your 16-character Google App Password", 857 key="b2b_gmail_session_password", 858)</code>	Renders a single-line text input box on the page. The value the user enters/selects is stored in <code>gmail_session_password</code> for use later in the script.
	Continuation of the multi-line <code>st.text_input(...)</code> call above, contributing the text “Gmail App Password”.
	Continuation of the multi-line <code>st.text_input(...)</code> call above, contributing the text “password”.
	Continuation of the multi-line <code>st.text_input(...)</code> call above, contributing the text “Enter your 16-character Google App Password”.
	Continuation of the multi-line <code>st.text_input(...)</code> call above, contributing the text “b2b_gmail_session_password”.
	Closes the multi-line <code>st.text_input(...)</code> call opened above.
859	Blank line — vertical spacing added purely for readability; has no functional effect.
860 <code>st.markdown("#### 📧 Email Preview")</code>	Renders formatted Markdown/HTML text output on the page, labelled/showing “#### 📧 Email Preview”.
861	Blank line — vertical spacing added purely for readability; has no functional effect.
862 <code>def _render_email(template, lead):</code>	Defines the function <code>_render_email(...)</code> , which fills a cold-outreach email template with a specific lead's details.
863 <code>if not isinstance(lead, dict):</code>	Conditional check — runs the indented block below only when <code>not isinstance(lead, dict)</code> evaluates to true.
864 <code>return str(template)</code>	Returns <code>str(template)</code> as the result of the current function, ending its execution.
865 <code>recipient_name = (866 lead.get("agency_name") or lead.get("company_name") or 867 lead.get("company") or lead.get("business_name") or 868 lead.get("name") or "Agency" 869)</code>	Inside the current function/block, sets the variable <code>recipient_name</code> to the result of <code>(.</code>
	Continuation of the multi-line <code>`recipient_name`</code> assignment` above, contributing the text “agency_name”.
	Continuation of the multi-line <code>`recipient_name`</code> assignment` above, contributing the text “company”.
	Continuation of the multi-line <code>`recipient_name`</code> assignment` above, contributing the text “name”.
	Closes the multi-line <code>`recipient_name`</code> assignment` opened above.
870 <code>recipient_email = lead.get("email") or lead.get("email_address") or lead.get("contact_email") or ""</code>	Inside the current function/block, sets the variable <code>recipient_email</code> to the result of <code>`lead.get("email")</code> or <code>lead.get("email_address")</code> or <code>lead.get("contact_e...”</code> .
871 <code>rendered = str(template)</code>	Inside the current function/block, sets the variable <code>rendered</code> to the result of <code>str(template)</code> .
872 <code>replacements = { 873 "{agency_name}": str(recipient_name), 874 "{company_name}": str(recipient_name), 875 "{company}": str(recipient_name), 876 "{business_name}": str(recipient_name), 877 "{recipient}": str(recipient_name),</code>	Starts a dictionary mapping email-template placeholders (e.g. <code>{name}</code>) to the actual lead values that will replace them when the outreach email is rendered.
	Continuation of the multi-line <code>replacements</code> dictionary (email placeholder → lead value map) above, contributing the text “{agency_name}”.
	Continuation of the multi-line <code>replacements</code> dictionary (email placeholder → lead value map) above, contributing the text “{company_name}”.
	Continuation of the multi-line <code>replacements</code> dictionary (email placeholder → lead value map) above, contributing the text “{company}”.
	Continuation of the multi-line <code>replacements</code> dictionary (email placeholder → lead value map) above, contributing the text “{business_name}”.
	Continuation of the multi-line <code>replacements</code> dictionary (email placeholder → lead value map) above, contributing the text

		"{recipient}".
878	"{name}": str(recipient_name),	Continuation of the multi-line replacements dictionary (email placeholder → lead value map) above, contributing the text "{name}".
879	"{email}": str(recipient_email),	Continuation of the multi-line replacements dictionary (email placeholder → lead value map) above, contributing the text "{email}".
880	"{location}": str(lead.get("location", "")),	Continuation of the multi-line replacements dictionary (email placeholder → lead value map) above, contributing the text "{location}".
881	}	Closes the multi-line replacements dictionary (email placeholder → lead value map) opened above.
882	for placeholder, value in replacements.items():	Loop — iterates placeholder, value over replacements.items(), running the indented block once per item.
883	rendered = rendered.replace(placeholder, value)	Inside the current function/block, sets the variable rendered to the result of rendered.replace(placeholder, value).
884		Blank line — vertical spacing added purely for readability; has no functional effect.
885	<i># Only preview-link fix: ensure the live platform URL is used.</i>	Comment (not executed): "Only preview-link fix: ensure the live platform URL is used." — documents intent for the code that follows.
886	rendered = rendered.replace(	Inside the current function/block, sets the variable rendered to the result of rendered.replace(.
887	"https://detective-ai.streamlit.app",	Supplies the https entry ('//detective-ai.streamlit.app',) as part of the multi-line rendered assignment` started above.
888	PLATFORM_URL.rstrip("/")	Continuation of the multi-line `rendered assignment` above, contributing the text "/".
889	)	Closes the multi-line `rendered assignment` opened above.
890	return rendered	Returns rendered as the result of the current function, ending its execution.
891		Blank line — vertical spacing added purely for readability; has no functional effect.
892	if selected_leads:	Conditional check — runs the indented block below only when selected_leads evaluates to true.
893	preview_options = [_lead_label(lead, i) for i, lead in enumerate(selected_leads)]	Inside the current function/block, sets the variable preview_options to the result of [_lead_label(lead, i) for i, lead in enumerate(selected_leads)].
894	preview_label = st.selectbox("Preview email for", options=preview_options, key="outreach_preview_agency_label")	Renders a dropdown selection menu on the page, labelled/showing "Preview email for". The value the user enters/selects is stored in preview_label for use later in the script.
895	preview_index = preview_options.index(preview_label)	Inside the current function/block, sets the variable preview_index to the result of preview_options.index(preview_label).
896	preview_lead = selected_leads[preview_index]	Inside the current function/block, sets the variable preview_lead to the result of selected_leads[preview_index].
897	preview_email = preview_lead.get("email") or preview_lead.get("email_address") or preview_lead.get("contact_email") or ""	Inside the current function/block, sets the variable preview_email to the result of `preview_lead.get("email") or preview_lead.get("email_address") or pre...`.
898	st.text_input("To", value=str(preview_email), disabled=True, key="outreach_preview_to")	Renders a single-line text input box on the page, labelled/showing "To".
899	st.text_input("Agency / Recipient", value=preview_label, disabled=True, key="outreach_preview_agency")	Renders a single-line text input box on the page, labelled/showing "Agency / Recipient".
900	st.text_input("Subject", value=_render_email(COLD_EMAIL_SUBJECT, preview_lead), disabled=True, key="outreach_preview_subject")	Renders a single-line text input box on the page, labelled/showing "Subject".
901	st.text_area("Email Body", value=_render_email(COLD_EMAIL_BODY, preview_lead), height=300, disabled=True, key="outreach_preview_body")	Renders a multi-line text box on the page, labelled/showing "Email Body".
902	st.caption("Nothing is sent until you click the dispatch button below.")	Renders small helper/caption text on the page, labelled/showing "Nothing is sent until you click the dispatch button below.".
903		Blank line — vertical spacing added purely for readability; has no functional effect.
904	st.divider()	Renders a horizontal divider line on the page.
905	if selected_leads:	Conditional check — runs the indented block below only when selected_leads evaluates to true.
906	if st.button(f"📬 Dispatch Cold Emails to {len(selected_leads)} Selected Recipients", use_container_width=True, key="btn_dispatch_selected_emails"):	Conditional check — runs the indented block below only when `st.button(f"📬 Dispatch Cold Emails to {len(selected_leads)} Selected Recipients...` evaluates to true.
907	app_password = str(gmail_session_password or "").strip()	Inside the current function/block, sets the variable app_password to the result of `str(gmail_session_password or "").strip()`.
908	if not app_password:	Conditional check — runs the indented block below only when not app_password evaluates to true.
909	st.error("Enter the Gmail App Password above, then try again.")	Renders a red error message box on the page, labelled/showing "Enter the Gmail App Password above, then try again.".
910	else:	Fallback branch — runs when none of the preceding if / elif conditions above were true.
911	progress_box = st.empty()	Renders an empty placeholder container that can be updated later on the page. A handle to it is stored in progress_box so its contents can be replaced later (e.g. to show progress updates).
912	with st.spinner(f"Sending emails to {len(selected_leads)} selected recipients..."):	Shows a loading spinner ("Sending emails to {len(selected_leads)} selected ...") while the indented block runs, giving the user visual feedback that work is in progress.
913	try:	Begins a try block — the app attempts the risky operation below and will gracefully handle failures instead of crashing the UI.
914	dispatch_result = send_cold_emails(	Inside the current function/block, sets the variable dispatch_result to the result of send_cold_emails(.
915	selected_leads,	Continuation line of the multi-line `dispatch_result assignment` statement started above.
916	app_password,	Continuation line of the multi-line `dispatch_result assignment` statement started above.
917	COLD_EMAIL_SUBJECT,	Continuation line of the multi-line `dispatch_result assignment` statement started above.
918	COLD_EMAIL_BODY,	Continuation line of the multi-line `dispatch_result assignment` statement started above.
919	delay_seconds=5,	Continuation line of the multi-line `dispatch_result assignment` statement started above.
920	progress_callback=lambda i, total, agency, status, error="": progress_box.info(	Continuation line of the multi-line `dispatch_result assignment` statement started above.
921	f"Sending	Continuation of the multi-line `dispatch_result assignment` above, contributing the text "Sending {i}/{total}: {agency}

921	<code> {i}/{total}: {agency} — {status}"</code>	Continuation of the multi-line <code>dispatch_result</code> assignment` above, contributing the text <code>Sending {i}/{total}: {agency} — {status}"</code> .
922	<code> error else ""</code>	Continuation of the multi-line <code>dispatch_result</code> assignment` above, contributing the text <code>— {error}"</code> .
923	<code>),</code>	Closes the multi-line <code>dispatch_result</code> assignment` opened above.
924	<code>)</code>	Closes the multi-line <code>dispatch_result</code> assignment` opened above.
925	<code> successful = [x for x in dispatch_result if str(x.get("status", "")).upper() in {"SENT", "SUCCESS", "SUCCESSFUL"}]</code>	Inside the current function/block, sets the variable <code>successful</code> to the result of <code>`[x for x in dispatch_result if str(x.get("status", "").upper() in {"SENT", "SUCCESS", "SUCCESSFUL"}]</code> .
926	<code> failed = [x for x in dispatch_result if str(x.get("status", "").upper() not in {"SENT", "SUCCESS", "SUCCESSFUL"}]</code>	Inside the current function/block, sets the variable <code>failed</code> to the result of <code>`[x for x in dispatch_result if str(x.get("status", "").upper() not in {"SENT", "SUCCESS", "SUCCESSFUL"}]</code> .
927	<code> if successful:</code>	Conditional check — runs the indented block below only when <code>successful</code> evaluates to true.
928	<code> st.success(f"Successfully sent **{len(successful)}** email(s).")</code>	Renders a green success message box on the page, labelled/showing <code>"Successfully sent **{len(successful)}** email(s)."</code> .
929	<code> if failed:</code>	Conditional check — runs the indented block below only when <code>failed</code> evaluates to true.
930	<code> st.warning(f"*** {len(failed)}** email(s) were not sent.")</code>	Renders an amber warning message box on the page, labelled/showing <code>"***{len(failed)}** email(s) were not sent."</code> .
931	<code> for item in failed:</code>	Loop — iterates <code>item</code> over <code>failed</code> , running the indented block once per item.
932	<code> st.error(f" {item.get('agency_name', item.get('recipient', 'Unknown'))}: {item.get('error', 'Unknown error')}")</code>	Renders a red error message box on the page, labelled/showing <code>"agency_name"</code> .
933	<code> if not successful and not failed:</code>	Conditional check — runs the indented block below only when <code>not successful</code> and <code>not failed</code> evaluates to true.
934	<code> st.info("No emails were dispatched.")</code>	Renders a blue informational message box on the page, labelled/showing <code>"No emails were dispatched."</code> .
935		Blank line — vertical spacing added purely for readability; has no functional effect.
936	<code> # Persist dispatch results so the admin can see</code>	Comment (not executed): <code>"Persist dispatch results so the admin can see"</code> — documents intent for the code that follows.
937	<code> # which prospects have already been contacted.</code>	Comment (not executed): <code>"which prospects have already been contacted."</code> — documents intent for the code that follows.
938	<code> result_by_email = {</code>	Starts building the dictionary <code>result_by_email</code> (key/value pairs continue on the following indented lines).
939	<code> str(item.get("recipient", "")).strip().lower(): item</code>	Continuation of the multi-line <code>result_by_email</code> dictionary literal` above, contributing the text <code>"recipient"</code> .
940	<code> dispatch_result</code>	Continuation line of the multi-line <code>result_by_email</code> dictionary literal` statement started above.
941	<code> if</code>	Continuation of the multi-line <code>result_by_email</code> dictionary literal` above, contributing the text <code>"recipient"</code> .
942	<code> item.get("recipient")</code>	
943	<code> } changed = False</code>	Closes the multi-line <code>result_by_email</code> dictionary literal` opened above.
944	<code> for lead in saved_leads:</code>	Loop — iterates <code>lead</code> over <code>saved_leads</code> , running the indented block once per item.
945	<code> if not isinstance(lead, dict):</code>	Conditional check — runs the indented block below only when <code>not isinstance(lead, dict)</code> evaluates to true.
946	<code> continue</code>	Executes: <code>continue</code> — supporting statement within the current block.
947	<code> lead_email = str(lead.get("contact_email")</code>	Inside the current function/block, sets the variable <code>lead_email</code> to the result of <code>str(</code> .
948	<code> lead.get("contact_email")</code>	Continuation of the multi-line <code>lead_email</code> assignment` above, contributing the text <code>"contact_email"</code> .
949	<code> or lead.get("email")</code>	Continuation of the multi-line <code>lead_email</code> assignment` above, contributing the text <code>"email"</code> .
950	<code> or lead.get("email_address")</code>	Continuation of the multi-line <code>lead_email</code> assignment` above, contributing the text <code>"email_address"</code> .
951	<code> or ""</code>	Continuation line of the multi-line <code>lead_email</code> assignment` statement started above.
952	<code>).strip().lower()</code>	Continuation line of the multi-line <code>lead_email</code> assignment` statement started above.
953	<code> result_by_email.get(lead_email)</code>	Inside the current function/block, sets the variable <code>item</code> to the result of <code>result_by_email.get(lead_email)</code> .
954	<code> if not item:</code>	Conditional check — runs the indented block below only when <code>not item</code> evaluates to true.
955	<code> continue</code>	Executes: <code>continue</code> — supporting statement within the current block.
956		Blank line — vertical spacing added purely for readability; has no functional effect.
957	<code> new_status = str(item.get("status", "Prospect"))</code>	Inside the current function/block, sets the variable <code>new_status</code> to the result of <code>str(item.get("status", "Prospect"))</code> .
958	<code> if lead.get("status") != new_status:</code>	Conditional check — runs the indented block below only when <code>lead.get("status") != new_status</code> evaluates to true.
959	<code> lead["status"] = new_status</code>	Inside the current function/block, sets the variable <code>lead["status"]</code> to the result of <code>new_status</code> .
960	<code> changed = True</code>	Inside the current function/block, sets the variable <code>changed</code> to the result of <code>True</code> .
961		Blank line — vertical spacing added purely for readability; has no functional effect.
962	<code> if new_status == "SENT":</code>	Conditional check — runs the indented block below only when <code>new_status == "SENT"</code> evaluates to true.
963	<code> lead["last_contacted_at"] = time.strftime("%Y-%m-%d %H:%M:%S"</code>	Inside the current function/block, sets the variable <code>lead["last_contacted_at"]</code> to the result of <code>time.strftime(</code> .
964	<code>)</code>	Continuation of the multi-line <code>lead["last_contacted_at"]</code> assignment` above, contributing the text <code>"%Y-%m-%d %H:%M:%S"</code> .
965	<code> changed = True</code>	Closes the multi-line <code>lead["last_contacted_at"]</code> assignment` opened above.
966		Blank line — vertical spacing added purely for readability; has no functional effect.
967		Blank line — vertical spacing added purely for readability; has no functional effect.

968	if changed:	Conditional check — runs the indented block below only when <code>changed</code> evaluates to true.
969	save_leads(saved_leads)	Calls <code>save_leads(...)</code> , executing that function/method for its side effect (its return value, if any, is not stored).
970		Blank line — vertical spacing added purely for readability; has no functional effect.
971	except Exception as e:	Catches an error from the <code>try</code> block above of type <code>Exception</code> as <code>e</code> , so the app can show a friendly message instead of crashing.
972	st.error(f"Email dispatch failed: {e}")	Renders a red error message box on the page, labelled/showing "Email dispatch failed: {e}".
973	else:	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
974	st.button("📧 Dispatch Cold Emails", use_container_width=True, key="btn_dispatch_disabled", disabled=True)	Renders a clickable button on the page, labelled/showing "📧 Dispatch Cold Emails".
975	else:	Fallback branch — runs when none of the preceding <code>if / elif</code> conditions above were true.
976	st.info("No saved leads available to email. Perform a lead scrape first.")	Renders a blue informational message box on the page, labelled/showing "No saved leads available to email. Perform a lead scrape fi...".

Generated walkthrough — every line of app.py (lines 1-976) is represented above with no gaps.